

Machine Learning Assignment: 3

Store.py

```
"""
FMCG Store Network Generator
Generates 1,500-5,000 stores across
multiple retail formats
"""

import json
import random
import argparse
import logging
from datetime import datetime, timedelta
from typing import List, Dict

import psycopg2
from psycopg2.extras import execute_values
import pandas as pd

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[logging.FileHandler('store_generator.log'), logging.StreamHandler()]
)
logger = logging.getLogger(__name__)

class StoreGenerator:
    """Generate realistic retail network
    with multiple store formats"""

    STORE_TYPES = {
        'Hypermarket': {'cycle_days': 7,
                        'multiplier': 2.0, 'pct': 0.10},
        'Supermarket': {'cycle_days': 6,
                        'multiplier': 1.5, 'pct': 0.15},
        'Large': {'cycle_days': 5,
                  'multiplier': 1.0, 'pct': 0.22},
        'Medium': {'cycle_days': 5,
                   'multiplier': 0.8, 'pct': 0.28},
        'Small': {'cycle_days': 5,
                  'multiplier': 0.6, 'pct': 0.18},
        'Mobile': {'cycle_days': 7,
                   'multiplier': 0.4, 'pct': 0.07}
    }

    CITIES = [
        ('Mumbai', 'West', 'Maharashtra'),
        ('Delhi', 'North', 'Delhi'),
        ('Bangalore', 'South',
         'Karnataka'),
```

```
        ('Hyderabad', 'South',
         'Telangana'),
        ('Chennai', 'South', 'Tamil
         Nadu'),
        ('Kolkata', 'East', 'West
         Bengal'),
        ('Pune', 'West', 'Maharashtra'),
        ('Ahmedabad', 'West', 'Gujarat'),
        ('Jaipur', 'North', 'Rajasthan'),
        ('Lucknow', 'North', 'Uttar
         Pradesh')
    ]

    def __init__(self, total_stores: int =
    2000):
        self.total_stores = total_stores
        self.stores = []
        self.store_id_counter = 1

    def generate_stores(self) -> None:
        """Generate complete store
        network"""
        logger.info(f"Generating
        {self.total_stores} stores...")

        for store_type, config in
        self.STORE_TYPES.items():
            count = int(self.total_stores
            * config['pct'])
            logger.info(f" Generating
            {count} {store_type} stores...")

            for _ in range(count):
                city, region, state =
                random.choice(self.CITIES)

                store = {
                    'store_id':
                    self.store_id_counter,
                    'store_type':
                    store_type,
                    'cycle_days':
                    config['cycle_days'],
                    'avg_sales_multiplier':
                    config['multiplier'] * random.gauss(1.0,
                    0.05),
                    'city': city,
                    'region': region,
                    'state': state,
                    'pin_code':
                    random.randint(100000, 999999),
                    'area_type': 'Urban'
                    if random.random() < 0.85 else 'Semi-
                    Urban',
```

```

        'established_date':
(datetime(2010, 1, 1) +
timedelta(days=random.randint(0,
4700))).strftime('%Y-%m-%d')
    }
    self.stores.append(store)
    self.store_id_counter += 1

    logger.info(f"✓ Generated
{len(self.stores)} stores")

    def save_to_csv(self, filename: str =
'store.csv') -> None:
        """Save stores to CSV"""
        logger.info(f"Saving to
{filename}...")
        df = pd.DataFrame(self.stores)
        df.to_csv(filename, index=False)
        logger.info(f"✓ Saved
{len(self.stores)} stores")

    def save_to_database(self, db_config:
Dict) -> None:
        """Save to PostgreSQL"""
        logger.info("Saving stores to
database...")
        conn =
psycopg2.connect(**db_config)
        cursor = conn.cursor()

        insert_query = """
            INSERT INTO stores (store_id,
store_type, cycle_days,
avg_sales_multiplier,
                                city,
region, state, pin_code, area_type,
established_date)
            VALUES %s
        """

        values = [
            (s['store_id'],
s['store_type'], s['cycle_days'],
s['avg_sales_multiplier'],
                s['city'], s['region'],
s['state'], s['pin_code'], s['area_type'],
s['established_date'])
            for s in self.stores
        ]

        execute_values(cursor,
insert_query, values)
        conn.commit()

```

```

        cursor.close()
        conn.close()
        logger.info("✓ Stores saved to
database")

def main():
    parser =
argparse.ArgumentParser(description='Gener
ate FMCG store network')
    parser.add_argument('--stores',
type=int, default=2000, help='Total number
of stores')
    parser.add_argument('--output',
default='store.csv', help='Output CSV
file')
    parser.add_argument('--db-config',
default='db_config.json')
    parser.add_argument('--to-database',
action='store_true')

    args = parser.parse_args()

    generator =
StoreGenerator(args.stores)
    generator.generate_stores()
    generator.save_to_csv(args.output)

    if args.to_database:
        with open(args.db_config, 'r') as
f:
            db_config = json.load(f)
            generator.save_to_database(db_conf
ig)

    logger.info("✓ Store generation
complete!")

if __name__ == '__main__':
    main()

```

Product.py

```

"""
FMCG Product Catalog Generator
Generates 5,000+ PSKUs with 15-50 SKU variants each, totaling ~100K-
125K SKUs
Implements realistic FMCG product hierarchies with authentic Indian
market pack sizes
"""

```

```

import json
import random
import argparse
import logging
from typing import List, Dict, Tuple
from datetime import datetime

```

```

import psycopg2
from psycopg2.extras import execute_values
import pandas as pd
from tqdm import tqdm

# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler('product_generator.log'),
        logging.StreamHandler()
    ]
)
logger = logging.getLogger(__name__)

class ProductGenerator:
    """Generate comprehensive FMCG product catalog with PSKU-SKU
    hierarchy"""

    def __init__(self, brand_mapping_file: str =
'brand_category_mapping.json'):
        """Initialize with brand-category mappings"""
        with open(brand_mapping_file, 'r', encoding='utf-8') as f:
            data = json.load(f)
            self.brand_mappings = data['brand_category_mappings']
            self.brand_details = data['brand_details']

        self.products = []
        self.product_id_counter = 1
        self.psku_sequence = {}
        self.sku_sequence = {}

    def generate_psku_code(self, brand: str, category: str, sub_category:
str) -> str:
        """Generate unique PSKU code"""
        brand_abbr = self._get_brand_abbreviation(brand)
        category_code = self._get_category_code(category)
        subcat_code = self._get_subcategory_code(sub_category)

        key = f"{brand_abbr}_{category_code}_{subcat_code}"
        if key not in self.psku_sequence:
            self.psku_sequence[key] = 0
            self.psku_sequence[key] += 1

        return f"{key}_{self.psku_sequence[key]:04d}"

    def generate_sku_code(self, psku_code: str, weight_value: float,
weight_unit: str,
        count_value: int = None, variant: str = "") -> str:
        """Generate unique SKU code"""
        weight_str = f"{int(weight_value)}{weight_unit}" if weight_value else
""
        count_str = f"PC{count_value:03d}" if count_value else ""
        variant_str = f"_{variant.upper()}" if variant else ""

        base_key =
f"{psku_code}_VAR_{weight_str}_{count_str}{variant_str}"

        if base_key not in self.sku_sequence:
            self.sku_sequence[base_key] = 0
            self.sku_sequence[base_key] += 1

        return f"{base_key}_{self.sku_sequence[base_key]:04d}"

    @staticmethod

```

```

def _get_brand_abbreviation(brand: str) -> str:
    """Get brand abbreviation for code generation"""
    abbr_map = {
        'Nestlé': 'NEST', 'Unilever': 'UNIL', 'ITC': 'ITC',
        'Procter & Gamble': 'PNG', 'Johnson & Johnson': 'JNJ',
        'Britannia': 'BRIT', 'Godrej': 'GODR', 'PepsiCo': 'PEPS',
        'Parle': 'PARL', 'Dabur': 'DABU'
    }
    return abbr_map.get(brand, brand[:4].upper())

    @staticmethod
    def _get_category_code(category: str) -> str:
        """Get category code"""
        code_map = {
            'Food & Beverages': 'FB', 'Personal Care': 'PC', 'Home Care': 'HC',
            'Snacks': 'SN', 'Baby & Childcare': 'BC', 'Health': 'HE'
        }
        return code_map.get(category, 'XX')

    @staticmethod
    def _get_subcategory_code(sub_category: str) -> str:
        """Get abbreviated subcategory code"""
        # Simplified mapping - can be expanded
        code_map = {
            'Instant Coffee': 'INSTCOF', 'Black Tea': 'TEA', 'Instant Noodles':
'INSTNOO',
            'Chocolate Confectionery': 'CHOC', 'Glucose Biscuits': 'BISC',
            'Wheat Flour': 'ATTA',
            'Bathing Soap': 'SOAP', 'Premium Soap': 'PSOAP', 'Shampoo':
'SHAMP',
            'Toothpaste': 'TOOTH', 'Face Cream': 'FACECR', 'Deodorant Spray':
'DEO',
            'Detergent Powder': 'DETPOW', 'Fabric Softener': 'FABSOFT',
            'Dishwash Gel': 'DISHGEL',
            'Potato Chips': 'CHIPS', 'Ice Cream': 'ICECREAM', 'Diapers
Medium': 'DIAPER',
            'Baby Soap': 'BSOAP', 'Baby Oil': 'BOIL', 'Notebooks': 'NOTE',
            'Adhesive Plasters': 'PLASTER', 'Mouthwash': 'MOUTHW', 'Hand
Sanitizer': 'SANIT',
            'Honey': 'HONEY', 'Health Supplements': 'SUPPL'
        }
        return code_map.get(sub_category,
sub_category[:8].upper().replace(' ', ''))

    def generate_food_beverages_products(self, target_pskus: int =
2100) -> None:
        """Generate Food & Beverages category products"""
        logger.info(f"Generating {target_pskus} Food & Beverages PSKUs...")

        # Nestlé products
        self._generate_coffee_products('Nestlé', 'Nescafe',
target_pskus=80)
        self._generate_noodles_products('Nestlé', 'Maggi',
target_pskus=60)
        self._generate_chocolate_products('Nestlé', 'KitKat',
target_pskus=40)
        self._generate_dairy_products('Nestlé', 'a+', target_pskus=50)

        # Unilever products
        self._generate_tea_products('Unilever', 'Brooke Bond',
target_pskus=70)
        self._generate_condiments_products('Unilever', 'Kissan',
target_pskus=50)

        # ITC products
        self._generate_atta_products('ITC', 'Aashirvaad', target_pskus=80)
        self._generate_biscuits_products('ITC', 'Sunfeast',
target_pskus=120)

```

```

self._generate_snacks_products('ITC', 'Bingo', target_pskus=90)
self._generate_noodles_products('ITC', 'YiPPee', target_pskus=40)
self._generate_juice_products('ITC', 'B Natural', target_pskus=60)

# Britannia products
self._generate_biscuits_products('Britannia', 'Good Day',
target_pskus=100)
self._generate_biscuits_products('Britannia', 'Marie Gold',
target_pskus=80)

# PepsiCo products
self._generate_beverages_products('PepsiCo', 'Pepsi',
target_pskus=60)
self._generate_juice_products('PepsiCo', 'Tropicana',
target_pskus=50)
self._generate_cereals_products('PepsiCo', 'Quaker',
target_pskus=40)

# Parle products
self._generate_parleg_products('Parle', 'Parle-G', target_pskus=60)

# Fill remaining with variety
current_count = len([p for p in self.products if p['category'] == 'Food
& Beverages'])
remaining = target_pskus - current_count // 22 # Approximate
PSKUs

if remaining > 0:
    self._generate_misc_food_products(remaining)

def _generate_coffee_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate instant coffee products with realistic variants"""
    variants = [
        ('Classic Instant Coffee', [5, 10, 25, 50, 100, 200], [3, 6, 14, 110,
200, 380]),
        ('Gold Premium Instant Coffee', [10, 25, 50, 100, 200], [8, 18, 125,
240, 460]),
        ('Sunrise Chicory Blend Coffee', [10, 25, 50, 100, 200], [5, 11, 95,
175, 335])
    ]

    for psku_name, sizes, base_prices in variants:
        psku = f"{sub_brand}{psku_name}"
        psku_code = self.generate_psku_code(brand, 'Food & Beverages',
'Instant Coffee')

        for size, base_price in zip(sizes, base_prices):
            price = base_price * random.gauss(1.0, 0.10)
            unit = 'G' if size < 1000 else 'KG'
            weight_val = size if size < 1000 else size / 1000

            sku = f"{psku}{size}g {'Sachet' if size <= 10 else 'Jar'}"
            sku_code = self.generate_sku_code(psku_code, size, 'G')

        self.products.append({
            'product_id': self.product_id_counter,
            'category': 'Food & Beverages',
            'sub_category': 'Instant Coffee',
            'brand': brand,
            'sub_brand': sub_brand,
            'psku': psku,
            'psku_code': psku_code,
            'sku': sku,
            'sku_code': sku_code,
            'weight_value': size,
            'weight_unit': 'G',
            'count_value': None,
            'unit_price': round(price, 2)
        })
        self.product_id_counter += 1

def _generate_noodles_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate instant noodles with multiple flavors"""
    base_variants = [
        ('Masala 2-Minute Noodles', [70, 140, 280, 560]),
        ('Chicken 2-Minute Noodles', [70, 280, 560]),
        ('Atta Noodles', [70, 280]),
        ('Veg Mania Noodles', [70, 280])
    ]

    for psku_name, sizes in base_variants:
        psku = f"{sub_brand}{psku_name}"
        psku_code = self.generate_psku_code(brand, 'Food & Beverages',
'Instant Noodles')

        for size in sizes:
            base_price = size * 0.4 if size == 70 else size * 0.35
            price = base_price * random.gauss(1.0, 0.10)

            sku = f"{psku}{size}g {'Single Pack' if size <= 70 else 'Family
Pack'}"
            sku_code = self.generate_sku_code(psku_code, size, 'G')

        self.products.append({
            'product_id': self.product_id_counter,
            'category': 'Food & Beverages',
            'sub_category': 'Instant Noodles',
            'brand': brand,
            'sub_brand': sub_brand,
            'psku': psku,
            'psku_code': psku_code,
            'sku': sku,
            'sku_code': sku_code,
            'weight_value': size,
            'weight_unit': 'G',
            'count_value': None,
            'unit_price': round(price, 2)
        })
        self.product_id_counter += 1

def _generate_chocolate_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate chocolate confectionery"""
    variants = [
        ('2-Finger Chocolate Bar', [18.5, 37], [10, 20]),
        ('4-Finger Chocolate Bar', [37, 45], [20, 25]),
        ('Chunky Chocolate Bar', [40, 50], [25, 30])
    ]

    for psku_name, sizes, base_prices in variants:
        psku = f"{sub_brand}{psku_name}"
        psku_code = self.generate_psku_code(brand, 'Food & Beverages',
'Chocolate Confectionery')

        for size, base_price in zip(sizes, base_prices):
            price = base_price * random.gauss(1.0, 0.10)

            sku = f"{psku}{size}g"
            sku_code = self.generate_sku_code(psku_code, size, 'G')

        self.products.append({
            'product_id': self.product_id_counter,
            'category': 'Food & Beverages',
            'sub_category': 'Chocolate Confectionery',

```

```

        'brand': brand,
        'sub_brand': sub_brand,
        'psku': psku,
        'psku_code': psku_code,
        'sku': sku,
        'sku_code': sku_code,
        'weight_value': size,
        'weight_unit': 'G',
        'count_value': None,
        'unit_price': round(price, 2)
    })
    self.product_id_counter += 1

def _generate_dairy_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate dairy products"""
    variants = [
        ('Toned Milk', [500, 1000], [25, 48]),
        ('Full Cream Milk', [500, 1000], [28, 54]),
        ('Paneer', [200], [90])
    ]

    for psku_name, sizes, base_prices in variants:
        psku = f"{sub_brand}{psku_name}"
        psku_code = self.generate_psku_code(brand, 'Food & Beverages',
'Dairy Products')

        for size, base_price in zip(sizes, base_prices):
            price = base_price * random.gauss(1.0, 0.08)
            unit = 'ML' if 'Milk' in psku_name else 'G'

            sku = f"{psku}{size}{unit.lower()}"
            sku_code = self.generate_sku_code(psku_code, size, unit)

            self.products.append({
                'product_id': self.product_id_counter,
                'category': 'Food & Beverages',
                'sub_category': 'Dairy Products',
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,
                'psku_code': psku_code,
                'sku': sku,
                'sku_code': sku_code,
                'weight_value': size,
                'weight_unit': unit,
                'count_value': None,
                'unit_price': round(price, 2)
            })
            self.product_id_counter += 1

def _generate_tea_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate tea products"""
    variants = [
        ('Red Label Black Tea', [100, 250, 500, 1000], [40, 95, 185, 360]),
        ('Taj Mahal Premium Tea', [100, 250, 500, 1000], [50, 120, 235,
460])
    ]

    for psku_name, sizes, base_prices in variants:
        psku = f"{sub_brand}{psku_name}"
        psku_code = self.generate_psku_code(brand, 'Food & Beverages',
'Black Tea')

        for size, base_price in zip(sizes, base_prices):
            price = base_price * random.gauss(1.0, 0.10)

```

```

            sku = f"{psku}{size}g"
            sku_code = self.generate_sku_code(psku_code, size, 'G')

            self.products.append({
                'product_id': self.product_id_counter,
                'category': 'Food & Beverages',
                'sub_category': 'Black Tea',
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,
                'psku_code': psku_code,
                'sku': sku,
                'sku_code': sku_code,
                'weight_value': size,
                'weight_unit': 'G',
                'count_value': None,
                'unit_price': round(price, 2)
            })
            self.product_id_counter += 1

def _generate_condiments_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate condiments"""
    variants = [
        ('Tomato Ketchup', [300, 500, 950], [45, 70, 125]),
        ('Mixed Fruit Jam', [200, 500], [80, 180])
    ]

    for psku_name, sizes, base_prices in variants:
        psku = f"{sub_brand}{psku_name}"
        psku_code = self.generate_psku_code(brand, 'Food & Beverages',
'Condiments')

        for size, base_price in zip(sizes, base_prices):
            price = base_price * random.gauss(1.0, 0.10)

            sku = f"{psku}{size}g"
            sku_code = self.generate_sku_code(psku_code, size, 'G')

            self.products.append({
                'product_id': self.product_id_counter,
                'category': 'Food & Beverages',
                'sub_category': 'Condiments',
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,
                'psku_code': psku_code,
                'sku': sku,
                'sku_code': sku_code,
                'weight_value': size,
                'weight_unit': 'G',
                'count_value': None,
                'unit_price': round(price, 2)
            })
            self.product_id_counter += 1

def _generate_atta_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate atta/flour products"""
    variants = [
        ('Shudh Chakki Atta', [1000, 2000, 5000, 10000], [55, 105, 255,
495]),
        ('Multigrain Atta', [1000, 5000], [75, 360]),
        ('Select Sharbati Atta', [1000, 5000], [80, 385])
    ]

    for psku_name, sizes, base_prices in variants:
        psku = f"{sub_brand}{psku_name}"

```

```

        psku_code = self.generate_psku_code(brand, 'Food & Beverages',
        'Wheat Flour')

        for size, base_price in zip(sizes, base_prices):
            price = base_price * random.gauss(1.0, 0.08)

            sku = f"{psku}{size}g"
            sku_code = self.generate_sku_code(psku_code, size, 'G')

            self.products.append({
                'product_id': self.product_id_counter,
                'category': 'Food & Beverages',
                'sub_category': 'Wheat Flour',
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,
                'psku_code': psku_code,
                'sku': sku,
                'sku_code': sku_code,
                'weight_value': size,
                'weight_unit': 'G',
                'count_value': None,
                'unit_price': round(price, 2)
            })
            self.product_id_counter += 1

    def _generate_biscuits_products(self, brand: str, sub_brand: str,
    target_pskus: int) -> None:
        """Generate biscuits products with multiple variants"""
        if sub_brand == 'Sunfeast':
            variants = [
                ('Dark Fantasy Choco Fills', [75, 150, 300, 600], [30, 58, 110,
                210]),
                ('Marie Light', [120, 250, 600], [25, 50, 115]),
                ('Bounce Creme Biscuit', [82, 150], [20, 36])
            ]
        elif sub_brand == 'Good Day':
            variants = [
                ('Butter Cookies', [75, 150, 300, 600], [25, 48, 92, 180]),
                ('Cashew Cookies', [75, 150], [30, 58])
            ]
        else:
            variants = [
                ('Classic Biscuits', [75, 150, 300], [22, 42, 80])
            ]

        for psku_name, sizes, base_prices in variants:
            psku = f"{sub_brand}{psku_name}"
            psku_code = self.generate_psku_code(brand, 'Food & Beverages',
            'Cream Biscuits')

            for size, base_price in zip(sizes, base_prices):
                price = base_price * random.gauss(1.0, 0.10)

                sku = f"{psku}{size}g"
                sku_code = self.generate_sku_code(psku_code, size, 'G')

                self.products.append({
                    'product_id': self.product_id_counter,
                    'category': 'Food & Beverages',
                    'sub_category': 'Cream Biscuits',
                    'brand': brand,
                    'sub_brand': sub_brand,
                    'psku': psku,
                    'psku_code': psku_code,
                    'sku': sku,
                    'sku_code': sku_code,
                    'weight_value': size,
                    'weight_unit': 'G',
                    'count_value': None,
                    'unit_price': round(price, 2)
                })
                self.product_id_counter += 1

    def _generate_snacks_products(self, brand: str, sub_brand: str,
    target_pskus: int) -> None:
        """Generate snacks products"""
        variants = [
            ('Mad Angles', [42, 90, 150], [20, 40, 65]),
            ('Tedhe Medhe', [35, 90], [15, 38])
        ]

        for psku_name, sizes, base_prices in variants:
            psku = f"{sub_brand}{psku_name}"
            psku_code = self.generate_psku_code(brand, 'Food & Beverages',
            'Ethnic Snacks')

            for size, base_price in zip(sizes, base_prices):
                price = base_price * random.gauss(1.0, 0.10)

                sku = f"{psku}{size}g"
                sku_code = self.generate_sku_code(psku_code, size, 'G')

                self.products.append({
                    'product_id': self.product_id_counter,
                    'category': 'Food & Beverages',
                    'sub_category': 'Ethnic Snacks',
                    'brand': brand,
                    'sub_brand': sub_brand,
                    'psku': psku,
                    'psku_code': psku_code,
                    'sku': sku,
                    'sku_code': sku_code,
                    'weight_value': size,
                    'weight_unit': 'G',
                    'count_value': None,
                    'unit_price': round(price, 2)
                })
                self.product_id_counter += 1

    def _generate_juice_products(self, brand: str, sub_brand: str,
    target_pskus: int) -> None:
        """Generate juice products"""
        variants = [
            ('Mixed Fruit Juice', [200, 500, 1000], [20, 45, 85]),
            ('Mango Juice', [200, 500, 1000], [22, 48, 90])
        ]

        for psku_name, sizes, base_prices in variants:
            psku = f"{sub_brand}{psku_name}"
            psku_code = self.generate_psku_code(brand, 'Food & Beverages',
            'Fruit Juice')

            for size, base_price in zip(sizes, base_prices):
                price = base_price * random.gauss(1.0, 0.10)

                sku = f"{psku}{size}ml Tetra Pack"
                sku_code = self.generate_sku_code(psku_code, size, 'ML')

                self.products.append({
                    'product_id': self.product_id_counter,
                    'category': 'Food & Beverages',
                    'sub_category': 'Fruit Juice',
                    'brand': brand,
                    'sub_brand': sub_brand,
                    'psku': psku,
                    'psku_code': psku_code,
                    'sku': sku,
                    'sku_code': sku_code,
                    'weight_value': size,
                    'weight_unit': 'G',
                    'count_value': None,
                    'unit_price': round(price, 2)
                })
                self.product_id_counter += 1

```

```

        'psku_code': psku_code,
        'sku': sku,
        'sku_code': sku_code,
        'weight_value': size,
        'weight_unit': 'ML',
        'count_value': None,
        'unit_price': round(price, 2)
    })
    self.product_id_counter += 1

def _generate_beverages_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate carbonated beverages"""
    variants = [
        ('Cola', [200, 500, 1000, 2250], [20, 40, 70, 120]),
        ('Diet Cola', [500, 1000], [45, 75])
    ]

    for psku_name, sizes, base_prices in variants:
        psku = f"{sub_brand}{psku_name}"
        psku_code = self.generate_psku_code(brand, 'Food & Beverages',
'Carbonated Drinks')

        for size, base_price in zip(sizes, base_prices):
            price = base_price * random.gauss(1.0, 0.10)

            sku = f"{psku}{size}ml Bottle"
            sku_code = self.generate_sku_code(psku_code, size, 'ML')

            self.products.append({
                'product_id': self.product_id_counter,
                'category': 'Food & Beverages',
                'sub_category': 'Carbonated Drinks',
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,
                'psku_code': psku_code,
                'sku': sku,
                'sku_code': sku_code,
                'weight_value': size,
                'weight_unit': 'ML',
                'count_value': None,
                'unit_price': round(price, 2)
            })
            self.product_id_counter += 1

def _generate_cereals_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate breakfast cereals"""
    variants = [
        ('Oats', [200, 500, 1000, 1500], [45, 105, 195, 280]),
        ('Oats Masala', [400, 1000], [95, 220])
    ]

    for psku_name, sizes, base_prices in variants:
        psku = f"{sub_brand}{psku_name}"
        psku_code = self.generate_psku_code(brand, 'Food & Beverages',
'Breakfast Cereals')

        for size, base_price in zip(sizes, base_prices):
            price = base_price * random.gauss(1.0, 0.10)

            sku = f"{psku}{size}g"
            sku_code = self.generate_sku_code(psku_code, size, 'G')

            self.products.append({
                'product_id': self.product_id_counter,
                'category': 'Food & Beverages',

```

```

                'sub_category': 'Breakfast Cereals',
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,
                'psku_code': psku_code,
                'sku': sku,
                'sku_code': sku_code,
                'weight_value': size,
                'weight_unit': 'G',
                'count_value': None,
                'unit_price': round(price, 2)
            })
            self.product_id_counter += 1

def _generate_parleg_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate Parle-G biscuits with extensive pack size range"""
    sizes = [5, 28, 45, 80, 180, 376, 799, 1000]
    base_prices = [2, 5, 10, 18, 38, 75, 155, 190]

    psku = f"{sub_brand} Original Glucose Biscuits"
    psku_code = self.generate_psku_code(brand, 'Food & Beverages',
'Glucose Biscuits')

    for size, base_price in zip(sizes, base_prices):
        price = base_price * random.gauss(1.0, 0.10)

        sku = f"{psku}{size}g"
        sku_code = self.generate_sku_code(psku_code, size, 'G')

        self.products.append({
            'product_id': self.product_id_counter,
            'category': 'Food & Beverages',
            'sub_category': 'Glucose Biscuits',
            'brand': brand,
            'sub_brand': sub_brand,
            'psku': psku,
            'psku_code': psku_code,
            'sku': sku,
            'sku_code': sku_code,
            'weight_value': size,
            'weight_unit': 'G',
            'count_value': None,
            'unit_price': round(price, 2)
        })
        self.product_id_counter += 1

def _generate_misc_food_products(self, target_pskus: int) -> None:
    """Generate additional food products to reach target"""
    logger.info(f"Generating {target_pskus} miscellaneous Food &
Beverages PSKUs...")

    # Generic product templates for variety (only brands allowed in
Food & Beverages)
    brands_subcats = [
        ('Nestlé', 'Cerelac', 'Baby Cereals'),
        ('Nestlé', 'Milo', 'Health Drinks'),
        ('Unilever', 'Horlicks', 'Health Drinks'),
        ('Unilever', 'Knorr', 'Soups'),
        ('Unilever', 'Bru', 'Coffee'),
        ('ITC', 'Candyman', 'Confectionery'),
        ('Britannia', 'Bourbon', 'Biscuits'),
        ('Britannia', 'Treat', 'Biscuits'),
        ('PepsiCo', 'Mountain Dew', 'Carbonated Drinks'),
        ('Parle', 'Monaco', 'Biscuits'),
        ('Parle', 'Krackjack', 'Crackers'),
    ]

```

```

sizes_weights = [50, 100, 200, 250, 500, 1000]

pskus_generated = 0
variant_num = 1

while pskus_generated < target_pskus:
    for brand, sub_brand, sub_category in brands_subcats:
        if pskus_generated >= target_pskus:
            break

        # Create a PSKU with multiple SKU variants
        psku = f"{sub_brand} Product Variant {variant_num}"
        psku_code = self.generate_psku_code(brand, 'Food & Beverages', sub_category)

        for size in random.sample(sizes_weights, k=random.randint(3, 6)):
            base_price = size * random.uniform(0.15, 0.50)
            price = base_price * random.gauss(1.0, 0.10)

            sku = f"{psku}{size}g"
            sku_code = self.generate_sku_code(psku_code, size, 'G')

            self.products.append({
                'product_id': self.product_id_counter,
                'category': 'Food & Beverages',
                'sub_category': sub_category,
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,
                'psku_code': psku_code,
                'sku': sku,
                'sku_code': sku_code,
                'weight_value': size,
                'weight_unit': 'G',
                'count_value': None,
                'unit_price': round(price, 2)
            })
            self.product_id_counter += 1

        pskus_generated += 1
        variant_num += 1

def generate_personal_care_products(self, target_pskus: int = 1300) -
> None:
    """Generate Personal Care category products"""
    logger.info(f"Generating {target_pskus} Personal Care PSKUs...")

    # Unilever products
    self._generate_soap_products('Unilever', 'Dove', 'Premium Soap',
target_pskus=50)
    self._generate_soap_products('Unilever', 'Lux', 'Bathing Soap',
target_pskus=50)
    self._generate_soap_products('Unilever', 'Lifebuoy', 'Bathing Soap',
target_pskus=40)
    self._generate_toothpaste_products('Unilever', 'Pepsodent',
target_pskus=40)
    self._generate_face_cream_products('Unilever', "Pond's",
target_pskus=40)

    # P&G products
    self._generate_shampoo_products('Procter & Gamble', 'Pantene',
target_pskus=60)
    self._generate_shampoo_products('Procter & Gamble', 'Head &
Shoulders', target_pskus=50)
    self._generate_razor_products('Procter & Gamble', 'Gillette',
target_pskus=40)

    # ITC products
    self._generate_soap_products('ITC', 'Fiams Di Wills', 'Premium
Soap', target_pskus=40)
    self._generate_deodorant_products('ITC', 'Engage',
target_pskus=50)

    # Godrej products
    self._generate_soap_products('Godrej', 'Godrej No.1', 'Bathing
Soap', target_pskus=30)
    self._generate_soap_products('Godrej', 'Cinthol', 'Bathing Soap',
target_pskus=30)

    # Dabur products
    self._generate_hair_oil_products('Dabur', 'Dabur Amla',
target_pskus=40)
    self._generate_toothpaste_products('Dabur', 'Dabur Red',
target_pskus=30)

    # Fill remaining with variety
    current_count = len([p for p in self.products if p['category'] ==
'Personal Care'])
    remaining = target_pskus - current_count // 4 # Approximate
PSKUs

    if remaining > 0:
        self._generate_misc_personal_care_products(remaining)

def _generate_soap_products(self, brand: str, sub_brand: str,
sub_category: str, target_pskus: int) -> None:
    """Generate soap products"""
    if sub_brand == 'Dove':
        variants = [
            ('Cream Beauty Bathing Bar', [50, 75, 100], [25, 35, 45]),
            ('Cream Beauty Bathing Bar 3-Pack', [300], [120])
        ]
    elif sub_brand == 'Lux':
        variants = [
            ('Velvet Touch Soap', [75, 100, 125], [20, 28, 35]),
            ('Velvet Touch Soap 4-Pack', [400], [115])
        ]
    elif sub_brand == 'Lifebuoy':
        variants = [
            ('Total Protection Soap', [100, 125], [25, 32]),
            ('Total Protection Soap 4-Pack', [400], [100])
        ]
    else:
        variants = [
            ('Bathing Bar', [75, 100, 125], [25, 32, 40])
        ]

    for psku_name, sizes, base_prices in variants:
        psku = f"{sub_brand}{psku_name}"
        psku_code = self.generate_psku_code(brand, 'Personal Care',
sub_category)

        for size, base_price in zip(sizes, base_prices):
            price = base_price * random.gauss(1.0, 0.10)

            sku = f"{psku}{size}g"
            sku_code = self.generate_sku_code(psku_code, size, 'G')

            self.products.append({
                'product_id': self.product_id_counter,
                'category': 'Personal Care',
                'sub_category': sub_category,
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,

```

```

        'psku_code': psku_code,
        'sku': sku,
        'sku_code': sku_code,
        'weight_value': size,
        'weight_unit': 'G',
        'count_value': None,
        'unit_price': round(price, 2)
    })
    self.product_id_counter += 1

def _generate_shampoo_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate shampoo products"""
    if sub_brand == 'Pantene':
        variants = [
            ('Pro-V Hair Fall Control Shampoo', [180, 340, 650, 1000], [180,
320, 580, 850])
        ]
    else:
        variants = [
            ('Anti-Dandruff Shampoo', [180, 340, 650], [190, 340, 620])
        ]

    for psku_name, sizes, base_prices in variants:
        psku = f"{sub_brand}{psku_name}"
        psku_code = self.generate_psku_code(brand, 'Personal Care',
'Shampoo')

        for size, base_price in zip(sizes, base_prices):
            price = base_price * random.gauss(1.0, 0.10)

            sku = f"{psku}{size}ml"
            sku_code = self.generate_sku_code(psku_code, size, 'ML')

            self.products.append({
                'product_id': self.product_id_counter,
                'category': 'Personal Care',
                'sub_category': 'Shampoo',
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,
                'psku_code': psku_code,
                'sku': sku,
                'sku_code': sku_code,
                'weight_value': size,
                'weight_unit': 'ML',
                'count_value': None,
                'unit_price': round(price, 2)
            })
            self.product_id_counter += 1

def _generate_toothpaste_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate toothpaste products"""
    sizes = [50, 80, 100, 150, 200, 300]
    base_prices = [30, 45, 55, 75, 95, 135]

    psku = f"{sub_brand} Toothpaste"
    psku_code = self.generate_psku_code(brand, 'Personal Care',
'Toothpaste')

    for size, base_price in zip(sizes, base_prices):
        price = base_price * random.gauss(1.0, 0.10)

        sku = f"{psku}{size}g"
        sku_code = self.generate_sku_code(psku_code, size, 'G')

        self.products.append({

```

```

                'product_id': self.product_id_counter,
                'category': 'Personal Care',
                'sub_category': 'Toothpaste',
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,
                'psku_code': psku_code,
                'sku': sku,
                'sku_code': sku_code,
                'weight_value': size,
                'weight_unit': 'G',
                'count_value': None,
                'unit_price': round(price, 2)
            })
            self.product_id_counter += 1

def _generate_face_cream_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate face cream products"""
    variants = [
        ('White Beauty Cream', [50, 100], [180, 340]),
        ('Age Miracle Cream', [50, 100], [220, 420])
    ]

    for psku_name, sizes, base_prices in variants:
        psku = f"{sub_brand}{psku_name}"
        psku_code = self.generate_psku_code(brand, 'Personal Care',
'Face Cream')

        for size, base_price in zip(sizes, base_prices):
            price = base_price * random.gauss(1.0, 0.10)

            sku = f"{psku}{size}g"
            sku_code = self.generate_sku_code(psku_code, size, 'G')

            self.products.append({
                'product_id': self.product_id_counter,
                'category': 'Personal Care',
                'sub_category': 'Face Cream',
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,
                'psku_code': psku_code,
                'sku': sku,
                'sku_code': sku_code,
                'weight_value': size,
                'weight_unit': 'G',
                'count_value': None,
                'unit_price': round(price, 2)
            })
            self.product_id_counter += 1

def _generate_razor_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate razor blade products"""
    variants = [
        ('Mach3 Turbo Razor Blades', [2, 4, 8, 12], [180, 340, 640, 920]),
        ('Fusion5 Razor Blades', [2, 4, 8], [240, 460, 880])
    ]

    for psku_name, counts, base_prices in variants:
        psku = f"{sub_brand}{psku_name}"
        psku_code = self.generate_psku_code(brand, 'Personal Care',
'Razor Blades')

        for count, base_price in zip(counts, base_prices):
            price = base_price * random.gauss(1.0, 0.10)

```

```

        sku = f"{psku}{count}-Count"
        sku_code = self.generate_sku_code(psku_code, 0, 'PC',
count_value=count)

        self.products.append({
            'product_id': self.product_id_counter,
            'category': 'Personal Care',
            'sub_category': 'Razor Blades',
            'brand': brand,
            'sub_brand': sub_brand,
            'psku': psku,
            'psku_code': psku_code,
            'sku': sku,
            'sku_code': sku_code,
            'weight_value': None,
            'weight_unit': 'PC',
            'count_value': count,
            'unit_price': round(price, 2)
        })
        self.product_id_counter += 1

    def _generate_deodorant_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
        """Generate deodorant products"""
        variants = [
            ('Deo Spray Men', [50, 100, 150], [90, 160, 220]),
            ('Deo Spray Women', [50, 100, 150], [95, 170, 240])
        ]

        for psku_name, sizes, base_prices in variants:
            psku = f"{sub_brand}{psku_name}"
            psku_code = self.generate_psku_code(brand, 'Personal Care',
'Deodorant Spray')

            for size, base_price in zip(sizes, base_prices):
                price = base_price * random.gauss(1.0, 0.10)

                sku = f"{psku}{size}ml"
                sku_code = self.generate_sku_code(psku_code, size, 'ML')

                self.products.append({
                    'product_id': self.product_id_counter,
                    'category': 'Personal Care',
                    'sub_category': 'Deodorant Spray',
                    'brand': brand,
                    'sub_brand': sub_brand,
                    'psku': psku,
                    'psku_code': psku_code,
                    'sku': sku,
                    'sku_code': sku_code,
                    'weight_value': size,
                    'weight_unit': 'ML',
                    'count_value': None,
                    'unit_price': round(price, 2)
                })
                self.product_id_counter += 1

    def _generate_hair_oil_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
        """Generate hair oil products"""
        sizes = [100, 200, 300, 550]
        base_prices = [75, 140, 200, 360]

        psku = f"{sub_brand} Hair Oil"
        psku_code = self.generate_psku_code(brand, 'Personal Care', 'Hair
Oil')

        for size, base_price in zip(sizes, base_prices):

```

```

            price = base_price * random.gauss(1.0, 0.10)

            sku = f"{psku}{size}ml"
            sku_code = self.generate_sku_code(psku_code, size, 'ML')

            self.products.append({
                'product_id': self.product_id_counter,
                'category': 'Personal Care',
                'sub_category': 'Hair Oil',
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,
                'psku_code': psku_code,
                'sku': sku,
                'sku_code': sku_code,
                'weight_value': size,
                'weight_unit': 'ML',
                'count_value': None,
                'unit_price': round(price, 2)
            })
            self.product_id_counter += 1

    def _generate_misc_personal_care_products(self, target_pskus: int) -
> None:
        """Generate additional personal care products to reach target"""
        logger.info(f"Generating {target_pskus} miscellaneous Personal
Care PSKUs...")

        brands_subcats = [
            ('Unilever', 'Vaseline', 'Skin Care'),
            ('Unilever', 'Lakmé', 'Cosmetics'),
            ('Procter & Gamble', 'Old Spice', 'Deodorant'),
            ('Procter & Gamble', 'Oral-B', 'Oral Care'),
            ('ITC', 'Vivel', 'Soap'),
            ('Godrej', 'Cinthol', 'Soap'),
            ('Dabur', 'Vatika', 'Hair Care'),
            ('Dabur', 'Babool', 'Oral Care'),
        ]

        sizes_weights = [50, 75, 100, 150, 200, 250]

        pskus_generated = 0
        variant_num = 1

        while pskus_generated < target_pskus:
            for brand, sub_brand, sub_category in brands_subcats:
                if pskus_generated >= target_pskus:
                    break

                psku = f"{sub_brand} Product Variant {variant_num}"
                psku_code = self.generate_psku_code(brand, 'Personal Care',
sub_category)

                for size in random.sample(sizes_weights, k=random.randint(3,
5)):
                    base_price = size * random.uniform(0.30, 0.80)
                    price = base_price * random.gauss(1.0, 0.10)

                    sku = f"{psku}{size}ml"
                    sku_code = self.generate_sku_code(psku_code, size, 'ML')

                    self.products.append({
                        'product_id': self.product_id_counter,
                        'category': 'Personal Care',
                        'sub_category': sub_category,
                        'brand': brand,
                        'sub_brand': sub_brand,
                        'psku': psku,

```

```

        'psku_code': psku_code,
        'sku': sku,
        'sku_code': sku_code,
        'weight_value': size,
        'weight_unit': 'ML',
        'count_value': None,
        'unit_price': round(price, 2)
    })
    self.product_id_counter += 1

    pskus_generated += 1
    variant_num += 1

def _generate_misc_home_care_products(self, target_pskus: int) ->
None:
    """Generate additional home care products to reach target"""
    logger.info(f"Generating {target_pskus} miscellaneous Home Care
    PSKUs...")

    brands_subcats = [
        ('Unilever', 'Domex', 'Toilet Cleaner'),
        ('Unilever', 'Rin', 'Detergent'),
        ('Procter & Gamble', 'Downy', 'Fabric Softener'),
        ('Procter & Gamble', 'Ambi Pur', 'Air Freshener'),
        ('ITC', 'Savlon', 'Disinfectant'),
    ]

    sizes_weights = [250, 500, 1000, 1500, 2000]

    pskus_generated = 0
    variant_num = 1

    while pskus_generated < target_pskus:
        for brand, sub_brand, sub_category in brands_subcats:
            if pskus_generated >= target_pskus:
                break

            psku = f"{sub_brand} Product Variant {variant_num}"
            psku_code = self.generate_psku_code(brand, 'Home Care',
            sub_category)

            for size in random.sample(sizes_weights, k=random.randint(3,
            5)):
                base_price = size * random.uniform(0.10, 0.30)
                price = base_price * random.gauss(1.0, 0.10)

                sku = f"{psku}{size}ml"
                sku_code = self.generate_sku_code(psku_code, size, 'ML')

                self.products.append({
                    'product_id': self.product_id_counter,
                    'category': 'Home Care',
                    'sub_category': sub_category,
                    'brand': brand,
                    'sub_brand': sub_brand,
                    'psku': psku,
                    'psku_code': psku_code,
                    'sku': sku,
                    'sku_code': sku_code,
                    'weight_value': size,
                    'weight_unit': 'ML',
                    'count_value': None,
                    'unit_price': round(price, 2)
                })
                self.product_id_counter += 1

            pskus_generated += 1
            variant_num += 1

```

```

def _generate_misc_snacks_products(self, target_pskus: int) ->
None:
    """Generate additional snacks products to reach target"""
    logger.info(f"Generating {target_pskus} miscellaneous Snacks
    PSKUs...")

    brands_subcats = [
        ('PepsiCo', 'Doritos', 'Chips'),
        ('PepsiCo', 'Kurkure', 'Snacks'),
        ('ITC', 'Bingo', 'Chips'),
        ('Parle', 'Hide & Seek', 'Biscuits'),
        ('Unilever', 'Cornetto', 'Ice Cream'),
    ]

    sizes_weights = [25, 50, 75, 100, 150, 200]

    pskus_generated = 0
    variant_num = 1

    while pskus_generated < target_pskus:
        for brand, sub_brand, sub_category in brands_subcats:
            if pskus_generated >= target_pskus:
                break

            psku = f"{sub_brand} Product Variant {variant_num}"
            psku_code = self.generate_psku_code(brand, 'Snacks',
            sub_category)

            for size in random.sample(sizes_weights, k=random.randint(3,
            5)):
                base_price = size * random.uniform(0.40, 1.00)
                price = base_price * random.gauss(1.0, 0.10)

                sku = f"{psku}{size}g"
                sku_code = self.generate_sku_code(psku_code, size, 'G')

                self.products.append({
                    'product_id': self.product_id_counter,
                    'category': 'Snacks',
                    'sub_category': sub_category,
                    'brand': brand,
                    'sub_brand': sub_brand,
                    'psku': psku,
                    'psku_code': psku_code,
                    'sku': sku,
                    'sku_code': sku_code,
                    'weight_value': size,
                    'weight_unit': 'G',
                    'count_value': None,
                    'unit_price': round(price, 2)
                })
                self.product_id_counter += 1

            pskus_generated += 1
            variant_num += 1

def _generate_misc_baby_childcare_products(self, target_pskus: int)
-> None:
    """Generate additional baby & childcare products to reach target"""
    logger.info(f"Generating {target_pskus} miscellaneous Baby &
    Childcare PSKUs...")

    brands_subcats = [
        ('Procter & Gamble', 'Pampers', 'Diapers'),
        ('Johnson & Johnson', 'Johnson's Baby', 'Baby Care'),
        ('ITC', 'Classmate', 'Stationery'),
    ]

```

```

        sizes_weights = [50, 100, 200, 250]

        pskus_generated = 0
        variant_num = 1

        while pskus_generated < target_pskus:
            for brand, sub_brand, sub_category in brands_subcats:
                if pskus_generated >= target_pskus:
                    break

                psku = f"{sub_brand} Product Variant {variant_num}"
                psku_code = self.generate_psku_code(brand, 'Baby &
Childcare', sub_category)

                for size in random.sample(sizes_weights, k=random.randint(3,
4)):
                    base_price = size * random.uniform(0.50, 1.50)
                    price = base_price * random.gauss(1.0, 0.10)

                    sku = f"{psku} {size}ml"
                    sku_code = self.generate_sku_code(psku_code, size, 'ML')

                    self.products.append({
                        'product_id': self.product_id_counter,
                        'category': 'Baby & Childcare',
                        'sub_category': sub_category,
                        'brand': brand,
                        'sub_brand': sub_brand,
                        'psku': psku,
                        'psku_code': psku_code,
                        'sku': sku,
                        'sku_code': sku_code,
                        'weight_value': size,
                        'weight_unit': 'ML',
                        'count_value': None,
                        'unit_price': round(price, 2)
                    })
                    self.product_id_counter += 1

                pskus_generated += 1
                variant_num += 1

        def generate_misc_health_products(self, target_pskus: int) -> None:
            """Generate additional health products to reach target"""
            logger.info(f"Generating {target_pskus} miscellaneous Health
PSKUs...")

            brands_subcats = [
                ('Johnson & Johnson', 'Band-Aid', 'First Aid'),
                ('Johnson & Johnson', 'Listerine', 'Mouthwash'),
                ('Dabur', 'Dabur Chyawanprash', 'Supplements'),
                ('Dabur', 'Dabur Pudina Hara', 'Digestive'),
                ('ITC', 'Savlon', 'Antiseptic'),
            ]

            sizes_weights = [50, 100, 200, 250, 500]

            pskus_generated = 0
            variant_num = 1

            while pskus_generated < target_pskus:
                for brand, sub_brand, sub_category in brands_subcats:
                    if pskus_generated >= target_pskus:
                        break

                    psku = f"{sub_brand} Product Variant {variant_num}"

                    psku_code = self.generate_psku_code(brand, 'Health',
sub_category)

                    for size in random.sample(sizes_weights, k=random.randint(3,
5)):
                        base_price = size * random.uniform(0.60, 1.50)
                        price = base_price * random.gauss(1.0, 0.10)

                        sku = f"{psku} {size}ml"
                        sku_code = self.generate_sku_code(psku_code, size, 'ML')

                        self.products.append({
                            'product_id': self.product_id_counter,
                            'category': 'Health',
                            'sub_category': sub_category,
                            'brand': brand,
                            'sub_brand': sub_brand,
                            'psku': psku,
                            'psku_code': psku_code,
                            'sku': sku,
                            'sku_code': sku_code,
                            'weight_value': size,
                            'weight_unit': 'ML',
                            'count_value': None,
                            'unit_price': round(price, 2)
                        })
                        self.product_id_counter += 1

                    pskus_generated += 1
                    variant_num += 1

            def generate_home_care_products(self, target_pskus: int = 700) ->
None:
                """Generate Home Care category products"""
                logger.info(f"Generating {target_pskus} Home Care PSKUs...")

                # Unilever products
                self._generate_detergent_products('Unilever', 'Surf Excel',
target_pskus=60)
                self._generate_detergent_products('Unilever', 'Rin',
target_pskus=50)
                self._generate_detergent_products('Unilever', 'Wheel',
target_pskus=40)
                self._generate_dishwash_products('Unilever', 'Vim',
target_pskus=40)
                self._generate_fabric_softener_products('Unilever', 'Comfort',
target_pskus=40)

                # P&G products
                self._generate_detergent_products('Procter & Gamble', 'Ariel',
target_pskus=60)
                self._generate_detergent_products('Procter & Gamble', 'Tide',
target_pskus=50)

                # Fill remaining with variety
                current_count = len([p for p in self.products if p['category'] ==
'Home Care'])
                remaining = target_pskus - current_count // 4

                if remaining > 0:
                    self._generate_misc_home_care_products(remaining)

            def _generate_detergent_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
                """Generate detergent powder products"""
                if sub_brand in ['Surf Excel', 'Ariel']:
                    sizes = [500, 1000, 2000, 4000, 6000]
                    base_prices = [100, 180, 340, 650, 950]

```

```

elif sub_brand == 'Tide':
    sizes = [500, 1000, 2000, 6000]
    base_prices = [95, 170, 320, 900]
else:
    sizes = [500, 1000, 2000, 4000]
    base_prices = [80, 140, 260, 480]

psku = f"{sub_brand} Detergent Powder"
psku_code = self.generate_psku_code(brand, 'Home Care',
'Detergent Powder')

for size, base_price in zip(sizes, base_prices):
    price = base_price * random.gauss(1.0, 0.10)

    sku = f"{psku}{size}g"
    sku_code = self.generate_sku_code(psku_code, size, 'G')

    self.products.append({
        'product_id': self.product_id_counter,
        'category': 'Home Care',
        'sub_category': 'Detergent Powder',
        'brand': brand,
        'sub_brand': sub_brand,
        'psku': psku,
        'psku_code': psku_code,
        'sku': sku,
        'sku_code': sku_code,
        'weight_value': size,
        'weight_unit': 'G',
        'count_value': None,
        'unit_price': round(price, 2)
    })
    self.product_id_counter += 1

def _generate_dishwash_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate dishwash products"""
    sizes = [250, 500, 750]
    base_prices = [50, 90, 125]

    psku = f"{sub_brand} Dishwash Gel"
    psku_code = self.generate_psku_code(brand, 'Home Care',
'Dishwash Gel')

    for size, base_price in zip(sizes, base_prices):
        price = base_price * random.gauss(1.0, 0.10)

        sku = f"{psku}{size}ml"
        sku_code = self.generate_sku_code(psku_code, size, 'ML')

        self.products.append({
            'product_id': self.product_id_counter,
            'category': 'Home Care',
            'sub_category': 'Dishwash Gel',
            'brand': brand,
            'sub_brand': sub_brand,
            'psku': psku,
            'psku_code': psku_code,
            'sku': sku,
            'sku_code': sku_code,
            'weight_value': size,
            'weight_unit': 'ML',
            'count_value': None,
            'unit_price': round(price, 2)
        })
        self.product_id_counter += 1

```

```

def _generate_fabric_softener_products(self, brand: str, sub_brand:
str, target_pskus: int) -> None:
    """Generate fabric softener products"""
    sizes = [200, 400, 800, 1600]
    base_prices = [45, 85, 160, 300]

    psku = f"{sub_brand} Fabric Softener"
    psku_code = self.generate_psku_code(brand, 'Home Care', 'Fabric
Softener')

    for size, base_price in zip(sizes, base_prices):
        price = base_price * random.gauss(1.0, 0.10)

        sku = f"{psku}{size}ml"
        sku_code = self.generate_sku_code(psku_code, size, 'ML')

        self.products.append({
            'product_id': self.product_id_counter,
            'category': 'Home Care',
            'sub_category': 'Fabric Softener',
            'brand': brand,
            'sub_brand': sub_brand,
            'psku': psku,
            'psku_code': psku_code,
            'sku': sku,
            'sku_code': sku_code,
            'weight_value': size,
            'weight_unit': 'ML',
            'count_value': None,
            'unit_price': round(price, 2)
        })
        self.product_id_counter += 1

def _generate_home_care_misc(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate miscellaneous home care products"""
    pass

def generate_snacks_products(self, target_pskus: int = 700) -> None:
    """Generate Snacks category products"""
    logger.info(f"Generating {target_pskus} Snacks PSKUs...")

    # PepsiCo Lay's chips
    self._generate_chips_products('PepsiCo', "Lay's",
target_pskus=100)
    self._generate_chips_products('PepsiCo', 'Kurkure',
target_pskus=60)

    # ITC snacks
    self._generate_snacks_products('ITC', 'Bingo', target_pskus=80)

    # Parle snacks
    self._generate_parle_snacks('Parle', 'Monaco', target_pskus=50)
    self._generate_parle_snacks('Parle', 'Hide & Seek',
target_pskus=50)

    # Unilever ice cream
    self._generate_icecream_products('Unilever', 'Kwality Walls',
target_pskus=60)

    # Fill remaining with variety
    current_count = len([p for p in self.products if p['category'] ==
'Snacks'])
    remaining = target_pskus - current_count // 4

    if remaining > 0:
        self._generate_misc_snacks_products(remaining)

```

```

def _generate_chips_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate potato chips with multiple flavors"""
    if sub_brand == "Lay's":
        flavors = ['Classic Salted', 'Cream & Onion', "India's Magic
Masala",
        'Spanish Tomato Tango', 'American Style Cream & Onion']
        sizes = [12, 24, 48, 52, 90, 145, 167]
        base_per_g = [0.83, 0.83, 0.83, 0.81, 0.78, 0.72, 0.70]
    else:
        flavors = ['Masala Munch', 'Chilli Chatka']
        sizes = [30, 50, 90, 150]
        base_per_g = [0.83, 0.80, 0.76, 0.72]

    for flavor in flavors:
        psku = f"{sub_brand}{flavor} Potato Chips"
        psku_code = self.generate_psku_code(brand, 'Snacks', 'Potato
Chips')

        for size, per_g in zip(sizes, base_per_g):
            price = size * per_g * random.gauss(1.0, 0.10)

            sku = f"{psku}{size}g"
            sku_code = self.generate_sku_code(psku_code, size, 'G',
variant=flavor[:3])

            self.products.append({
                'product_id': self.product_id_counter,
                'category': 'Snacks',
                'sub_category': 'Potato Chips',
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,
                'psku_code': psku_code,
                'sku': sku,
                'sku_code': sku_code,
                'weight_value': size,
                'weight_unit': 'G',
                'count_value': None,
                'unit_price': round(price, 2)
            })
            self.product_id_counter += 1

def _generate_parle_snacks(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate Parle snack biscuits"""
    if sub_brand == 'Monaco':
        sizes = [75, 150, 200, 400]
        base_prices = [15, 28, 36, 70]
    else:
        sizes = [70, 100, 150, 400]
        base_prices = [20, 28, 40, 105]

    psku = f"{sub_brand} Crackers"
    psku_code = self.generate_psku_code(brand, 'Snacks', 'Crackers')

    for size, base_price in zip(sizes, base_prices):
        price = base_price * random.gauss(1.0, 0.10)

        sku = f"{psku}{size}g"
        sku_code = self.generate_sku_code(psku_code, size, 'G')

        self.products.append({
            'product_id': self.product_id_counter,
            'category': 'Snacks',
            'sub_category': 'Crackers',
            'brand': brand,
            'sub_brand': sub_brand,

```

```

            'psku': psku,
            'psku_code': psku_code,
            'sku': sku,
            'sku_code': sku_code,
            'weight_value': size,
            'weight_unit': 'G',
            'count_value': None,
            'unit_price': round(price, 2)
        })
        self.product_id_counter += 1

def _generate_icecream_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate ice cream products"""
    variants = [
        ('Vanilla Ice Cream Tub', [100, 700], [40, 250]),
        ('Chocolate Ice Cream Tub', [100, 700], [45, 270]),
        ('Cornetto Cone', [125], [50])
    ]

    for psku_name, sizes, base_prices in variants:
        psku = f"{sub_brand}{psku_name}"
        psku_code = self.generate_psku_code(brand, 'Snacks', 'Ice
Cream')

        for size, base_price in zip(sizes, base_prices):
            price = base_price * random.gauss(1.0, 0.10)

            sku = f"{psku}{size}ml"
            sku_code = self.generate_sku_code(psku_code, size, 'ML')

            self.products.append({
                'product_id': self.product_id_counter,
                'category': 'Snacks',
                'sub_category': 'Ice Cream',
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,
                'psku_code': psku_code,
                'sku': sku,
                'sku_code': sku_code,
                'weight_value': size,
                'weight_unit': 'ML',
                'count_value': None,
                'unit_price': round(price, 2)
            })
            self.product_id_counter += 1

def generate_baby_childcare_products(self, target_pskus: int = 350) -
> None:
    """Generate Baby & Childcare category products"""
    logger.info(f"Generating {target_pskus} Baby & Childcare PSKUs...")

    # P&G Pampers diapers
    self._generate_diaper_products('Procter & Gamble', 'Pampers',
target_pskus=120)

    # J&J baby products
    self._generate_baby_care_products('Johnson & Johnson',
"Johnson's Baby", target_pskus=80)

    # ITC Classmate stationery
    self._generate_stationery_products('ITC', 'Classmate',
target_pskus=50)

    # Fill remaining with variety
    current_count = len([p for p in self.products if p['category'] == 'Baby
& Childcare'])

```

```

remaining = target_pskus - current_count // 3

if remaining > 0:
    self._generate_misc_baby_childcare_products(remaining)

def _generate_diaper_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate diaper products with count-based sizing"""
    diaper_types = [
        ('Newborn', [(20, 380), (42, 750)]),
        ('Small', [(46, 920), (76, 1450)]),
        ('Medium', [(52, 1150), (68, 1450), (124, 2550)]),
        ('Large', [(48, 1200), (62, 1550), (104, 2800)])
    ]

    for size_name, count_prices in diaper_types:
        psku = f"{sub_brand} Premium Care Diapers {size_name}"
        psku_code = self.generate_psku_code(brand, 'Baby & Childcare',
f'Diapers {size_name}')

        for count, base_price in count_prices:
            price = base_price * random.gauss(1.0, 0.08)

            sku = f"{psku}{count}-count"
            sku_code = self.generate_sku_code(psku_code, 0, 'PC',
count_value=count)

            self.products.append({
                'product_id': self.product_id_counter,
                'category': 'Baby & Childcare',
                'sub_category': f'Diapers {size_name}',
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,
                'psku_code': psku_code,
                'sku': sku,
                'sku_code': sku_code,
                'weight_value': None,
                'weight_unit': 'PC',
                'count_value': count,
                'unit_price': round(price, 2)
            })
            self.product_id_counter += 1

def _generate_baby_care_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate baby soap, oil, powder, etc."""
    variants = [
        ('Baby Soap', 'G', [75, 225], [45, 125]),
        ('Baby Oil', 'ML', [100, 200, 500], [120, 220, 510]),
        ('Baby Powder', 'G', [100, 200, 400], [150, 280, 530]),
        ('Baby Shampoo', 'ML', [100, 200, 500], [130, 240, 560])
    ]

    for psku_name, unit, sizes, base_prices in variants:
        psku = f"{sub_brand}{psku_name}"
        psku_code = self.generate_psku_code(brand, 'Baby & Childcare',
psku_name.replace(' ', ''))

        for size, base_price in zip(sizes, base_prices):
            price = base_price * random.gauss(1.0, 0.10)

            sku = f"{psku}{size}{unit.lower()}"
            sku_code = self.generate_sku_code(psku_code, size, unit)

            self.products.append({
                'product_id': self.product_id_counter,
                'category': 'Baby & Childcare',

```

```

                'sub_category': psku_name.replace(' ', ''),
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,
                'psku_code': psku_code,
                'sku': sku,
                'sku_code': sku_code,
                'weight_value': size,
                'weight_unit': unit,
                'count_value': None,
                'unit_price': round(price, 2)
            })
            self.product_id_counter += 1

def _generate_stationery_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate stationery products"""
    variants = [
        ('Notebook 172 Pages', [(1, 40), (5, 190), (10, 370)]),
        ('Notebook 240 Pages', [(1, 55), (5, 265)]),
        ('Ball Pen', [(1, 10), (5, 48), (10, 95)]),
        ('Pencil Pack', [(5, 25), (10, 48), (20, 92)])
    ]

    for psku_name, count_prices in variants:
        psku = f"{sub_brand}{psku_name}"
        psku_code = self.generate_psku_code(brand, 'Baby & Childcare',
'Notebooks')

        for count, base_price in count_prices:
            price = base_price * random.gauss(1.0, 0.10)

            sku = f"{psku}{count}-Count"
            sku_code = self.generate_sku_code(psku_code, 0, 'PC',
count_value=count)

            self.products.append({
                'product_id': self.product_id_counter,
                'category': 'Baby & Childcare',
                'sub_category': 'Notebooks',
                'brand': brand,
                'sub_brand': sub_brand,
                'psku': psku,
                'psku_code': psku_code,
                'sku': sku,
                'sku_code': sku_code,
                'weight_value': None,
                'weight_unit': 'PC',
                'count_value': count,
                'unit_price': round(price, 2)
            })
            self.product_id_counter += 1

def generate_health_products(self, target_pskus: int = 350) -> None:
    """Generate Health category products"""
    logger.info(f"Generating {target_pskus} Health PSKUs...")

    # J&J health products
    self._generate_firstaid_products('Johnson & Johnson', 'Band-Aid',
target_pskus=40)
    self._generate_mouthwash_products('Johnson & Johnson',
'Listerine', target_pskus=30)

    # ITC Savlon
    self._generate_sanitizer_products('ITC', 'Savlon', target_pskus=40)

    # Dabur health products
    self._generate_honey_products('Dabur', 'Dabur', target_pskus=50)

```

```

        self._generate_supplement_products('Dabur', 'Dabur',
target_pskus=50)

        # Fill remaining with variety
        current_count = len([p for p in self.products if p['category'] ==
'Health'])
        remaining = target_pskus - current_count // 4

        if remaining > 0:
            self._generate_misc_health_products(remaining)

        def _generate_firstaid_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
            """Generate adhesive plasters"""
            counts = [10, 20, 30, 50, 100]
            base_prices = [45, 85, 120, 190, 350]

            psku = f"{sub_brand} Adhesive Plasters"
            psku_code = self.generate_psku_code(brand, 'Health', 'Adhesive
Plasters')

            for count, base_price in zip(counts, base_prices):
                price = base_price * random.gauss(1.0, 0.10)

                sku = f"{psku} {count}-Count"
                sku_code = self.generate_sku_code(psku_code, 0, 'PC',
count_value=count)

                self.products.append({
                    'product_id': self.product_id_counter,
                    'category': 'Health',
                    'sub_category': 'Adhesive Plasters',
                    'brand': brand,
                    'sub_brand': sub_brand,
                    'psku': psku,
                    'psku_code': psku_code,
                    'sku': sku,
                    'sku_code': sku_code,
                    'weight_value': None,
                    'weight_unit': 'PC',
                    'count_value': count,
                    'unit_price': round(price, 2)
                })
                self.product_id_counter += 1

        def _generate_mouthwash_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
            """Generate mouthwash products"""
            sizes = [80, 250, 500, 1000]
            base_prices = [55, 130, 240, 450]

            psku = f"{sub_brand} Cool Mint Mouthwash"
            psku_code = self.generate_psku_code(brand, 'Health',
'Mouthwash')

            for size, base_price in zip(sizes, base_prices):
                price = base_price * random.gauss(1.0, 0.10)

                sku = f"{psku} {size}ml"
                sku_code = self.generate_sku_code(psku_code, size, 'ML')

                self.products.append({
                    'product_id': self.product_id_counter,
                    'category': 'Health',
                    'sub_category': 'Mouthwash',
                    'brand': brand,
                    'sub_brand': sub_brand,
                    'psku': psku,

```

```

                    'psku_code': psku_code,
                    'sku': sku,
                    'sku_code': sku_code,
                    'weight_value': size,
                    'weight_unit': 'ML',
                    'count_value': None,
                    'unit_price': round(price, 2)
                })
                self.product_id_counter += 1

        def _generate_sanitizer_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
            """Generate hand sanitizer products"""
            sizes = [50, 100, 200, 500]
            base_prices = [30, 55, 100, 240]

            psku = f"{sub_brand} Hand Sanitizer"
            psku_code = self.generate_psku_code(brand, 'Health', 'Hand
Sanitizer')

            for size, base_price in zip(sizes, base_prices):
                price = base_price * random.gauss(1.0, 0.10)

                sku = f"{psku} {size}ml"
                sku_code = self.generate_sku_code(psku_code, size, 'ML')

                self.products.append({
                    'product_id': self.product_id_counter,
                    'category': 'Health',
                    'sub_category': 'Hand Sanitizer',
                    'brand': brand,
                    'sub_brand': sub_brand,
                    'psku': psku,
                    'psku_code': psku_code,
                    'sku': sku,
                    'sku_code': sku_code,
                    'weight_value': size,
                    'weight_unit': 'ML',
                    'count_value': None,
                    'unit_price': round(price, 2)
                })
                self.product_id_counter += 1

        def _generate_honey_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
            """Generate honey products"""
            sizes = [100, 225, 250, 400, 500, 1000]
            base_prices = [70, 150, 165, 250, 310, 580]

            psku = f"{sub_brand} Honey"
            psku_code = self.generate_psku_code(brand, 'Health', 'Honey')

            for size, base_price in zip(sizes, base_prices):
                price = base_price * random.gauss(1.0, 0.10)

                sku = f"{psku} {size}g"
                sku_code = self.generate_sku_code(psku_code, size, 'G')

                self.products.append({
                    'product_id': self.product_id_counter,
                    'category': 'Health',
                    'sub_category': 'Honey',
                    'brand': brand,
                    'sub_brand': sub_brand,
                    'psku': psku,
                    'psku_code': psku_code,
                    'sku': sku,
                    'sku_code': sku_code,

```

```

        'weight_value': size,
        'weight_unit': 'G',
        'count_value': None,
        'unit_price': round(price, 2)
    })
    self.product_id_counter += 1

def _generate_supplement_products(self, brand: str, sub_brand: str,
target_pskus: int) -> None:
    """Generate health supplement products"""
    sizes = [250, 500, 1000]
    base_prices = [190, 360, 480]

    psku = f"{sub_brand} Chyawanprash"
    psku_code = self.generate_psku_code(brand, 'Health', 'Health
Supplements')

    for size, base_price in zip(sizes, base_prices):
        price = base_price * random.gauss(1.0, 0.10)

        sku = f"{psku} {size}g"
        sku_code = self.generate_sku_code(psku_code, size, 'G')

    self.products.append({
        'product_id': self.product_id_counter,
        'category': 'Health',
        'sub_category': 'Health Supplements',
        'brand': brand,
        'sub_brand': sub_brand,
        'psku': psku,
        'psku_code': psku_code,
        'sku': sku,
        'sku_code': sku_code,
        'weight_value': size,
        'weight_unit': 'G',
        'count_value': None,
        'unit_price': round(price, 2)
    })
    self.product_id_counter += 1

def validate_brand_category_mappings(self) -> bool:
    """Validate all products follow brand-category rules"""
    logger.info("Validating brand-category mappings...")
    violations = []

    for product in self.products:
        brand = product['brand']
        category = product['category']
        allowed_categories = self.brand_mappings.get(brand, [])

        if category not in allowed_categories:
            violations.append(
                f"Brand '{brand}' product in unauthorized category '{category}'"
            )
        else:
            violations.append(
                f"Product: {product['sku']}"
            )

    if violations:
        logger.error(f"Found {len(violations)} brand-category mapping
violations:")
        for v in violations[:10]:
            logger.error(f" {v}")
        return False

    logger.info("✓ All brand-category mappings valid")
    return True

def save_to_csv(self, filename: str = 'product.csv') -> None:

```

```

    """Save products to CSV file"""
    logger.info(f"Saving {len(self.products)} products to {filename}...")
    df = pd.DataFrame(self.products)
    df.to_csv(filename, index=False)
    logger.info(f"✓ Saved to {filename}")

def save_to_database(self, db_config: Dict) -> None:
    """Save products to PostgreSQL database"""
    logger.info(f"Saving {len(self.products)} products to PostgreSQL...")

    conn = psycopg2.connect(**db_config)
    cursor = conn.cursor()

    # Drop indexes for faster insertion
    logger.info("Dropping indexes for faster insertion...")
    cursor.execute("DROP INDEX IF EXISTS idx_products_psku")
    cursor.execute("DROP INDEX IF EXISTS
idx_products_category_brand")
    cursor.execute("DROP INDEX IF EXISTS idx_products_sku")
    cursor.execute("DROP INDEX IF EXISTS idx_products_brand")
    conn.commit()

    # Bulk insert
    logger.info("Bulk inserting products...")
    insert_query = """
        INSERT INTO products (
            product_id, category, sub_category, brand, sub_brand,
            psku, psku_code, sku, sku_code, weight_value, weight_unit,
            count_value, unit_price
        ) VALUES %s
    """

    values = [
        (
            p['product_id'], p['category'], p['sub_category'], p['brand'],
            p['sub_brand'],
            p['psku'], p['psku_code'], p['sku'], p['sku_code'],
            p['weight_value'],
            p['weight_unit'], p['count_value'], p['unit_price']
        )
        for p in self.products
    ]

    execute_values(cursor, insert_query, values, page_size=1000)
    conn.commit()

    # Recreate indexes
    logger.info("Recreating indexes...")
    cursor.execute("CREATE INDEX idx_products_psku ON
products(psku_code)")
    cursor.execute("CREATE INDEX idx_products_category_brand ON
products(category, brand)")
    cursor.execute("CREATE INDEX idx_products_sku ON
products(sku_code)")
    cursor.execute("CREATE INDEX idx_products_brand ON
products(brand)")
    conn.commit()

    cursor.close()
    conn.close()

    logger.info("✓ Products saved to database")

def generate_all_products(self) -> None:
    """Generate complete product catalog across all categories"""
    logger.info("Starting product catalog generation...")
    start_time = datetime.now()

```

```

self.generate_food_beverages_products(target_pskus=2100)
self.generate_personal_care_products(target_pskus=1300)
self.generate_home_care_products(target_pskus=700)
self.generate_snacks_products(target_pskus=700)
self.generate_baby_childcare_products(target_pskus=350)
self.generate_health_products(target_pskus=350)

# Validate
if not self.validate_brand_category_mappings():
    raise ValueError("Brand-category mapping validation failed!")

# Summary statistics
total_products = len(self.products)
total_pskus = len(set(p['psku_code'] for p in self.products))

logger.info(f"""
Product Catalog Generation Complete
=====
Total PSKUs: {total_pskus:,}
Total SKUs: {total_products:,}
Avg SKUs per PSKU: {total_products / total_pskus:.1f}
Generation Time: {(datetime.now() - start_time).total_seconds():.1f}s
""")

# Category breakdown
category_counts = {}
for p in self.products:
    cat = p['category']
    category_counts[cat] = category_counts.get(cat, 0) + 1

logger.info("Category Distribution:")
for cat, count in sorted(category_counts.items(), key=lambda x: -
x[1]):
    logger.info(f" {cat}: {count:,} SKUs")

def main():
    parser = argparse.ArgumentParser(
        description='Generate FMCG product catalog with PSKU-SKU
        hierarchy'
    )
    parser.add_argument('--output', default='product.csv', help='Output
    CSV file')
    parser.add_argument('--db-config', default='db_config.json',
    help='Database config file')
    parser.add_argument('--to-database', action='store_true', help='Save
    to PostgreSQL')
    parser.add_argument('--brand-mapping',
    default='brand_category_mapping.json',
        help='Brand-category mapping file')

    args = parser.parse_args()

    # Generate products
    generator = ProductGenerator(args.brand_mapping)
    generator.generate_all_products()

    # Save to CSV
    generator.save_to_csv(args.output)

    # Save to database if requested
    if args.to_database:
        with open(args.db_config, 'r') as f:
            db_config = json.load(f)
            generator.save_to_database(db_config)

    logger.info("✓ Product generation complete!")

```

```

if __name__ == '__main__':
    main()

```

Order.py

```

"""
FMCG Order Generator - Generates ~1
billion order records
Uses cycle-based ordering with Poisson-
distributed quantities
"""

import json
import argparse
import logging
import signal
import sys
from datetime import datetime, timedelta,
date
from typing import Dict, List, Tuple
import random

import psycopg2
from psycopg2.extras import execute_values
import numpy as np
from tqdm import tqdm

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s -
    %(message)s',
    handlers=[logging.FileHandler('order_g
    enerator.log'), logging.StreamHandler()]
)
logger = logging.getLogger(__name__)

class OrderGenerator:
    """Generate billion-scale order
    records with cycle-based logic"""

    def __init__(self, db_config: Dict,
    orders_per_cycle: int = 200):
        self.db_config = db_config
        self.orders_per_cycle =
    orders_per_cycle
        self.start_date = date(2023, 6, 1)
        self.end_date = date(2025, 12, 31)
        self.total_days = (self.end_date -
        self.start_date).days + 1

        self.conn = None
        self.cursor = None

```

```

        self.product_ids = []
        self.product_prices = {}
        self.stores = []
        self.store_start_dates = {}

        self.total_orders_inserted = 0
        self.checkpoint_file =
'order_checkpoint.json'
        self.last_processed_date = None

        # Setup signal handlers for
graceful shutdown
        signal.signal(signal.SIGINT,
self._signal_handler)
        signal.signal(signal.SIGTERM,
self._signal_handler)

        def _signal_handler(self, signum,
frame):
            """Handle graceful shutdown"""
            logger.warning("Received shutdown
signal, saving checkpoint...")
            self._save_checkpoint()
            if self.conn:
                self.conn.commit()
                self.conn.close()
            sys.exit(0)

        def _save_checkpoint(self):
            """Save current progress"""
            checkpoint = {
                'last_processed_date':
self.last_processed_date.isoformat() if
self.last_processed_date else None,
                'total_orders_inserted':
self.total_orders_inserted,
                'store_start_dates': {k:
v.isoformat() for k, v in
self.store_start_dates.items()}
            }
            with open(self.checkpoint_file,
'w') as f:
                json.dump(checkpoint, f)
                logger.info(f"Checkpoint saved:
{self.total_orders_inserted:,} orders
inserted")

        def _load_checkpoint(self) -> bool:
            """Load checkpoint if exists"""
            try:
                with
open(self.checkpoint_file, 'r') as f:
                    checkpoint = json.load(f)

```

```

                self.last_processed_date =
date.fromisoformat(checkpoint['last_proces
sed_date']) if
checkpoint['last_processed_date'] else
None

                self.total_orders_inserted =
checkpoint['total_orders_inserted']
                self.store_start_dates =
{int(k): date.fromisoformat(v) for k, v in
checkpoint['store_start_dates'].items()}
                logger.info(f"Resuming from
checkpoint: {self.total_orders_inserted:,}
orders already inserted")
                return True
            except FileNotFoundError:
                return False

        def connect_database(self):
            """Establish database
connection"""
            self.conn =
psycopg2.connect(**self.db_config)
            self.cursor = self.conn.cursor()
            logger.info("✓ Database
connected")

        def load_products(self):
            """Load product master data"""
            logger.info("Loading product
master data...")
            self.cursor.execute("SELECT
product_id, unit_price FROM products")
            rows = self.cursor.fetchall()

            self.product_ids = [row[0] for row
in rows]
            self.product_prices = {row[0]:
row[1] for row in rows}

            logger.info(f"✓ Loaded
{len(self.product_ids):,} products")

        def load_stores(self):
            """Load store master data"""
            logger.info("Loading store master
data...")
            self.cursor.execute("""
                SELECT store_id, cycle_days,
avg_sales_multiplier
                FROM stores
            """)

            self.stores = []
            for row in self.cursor.fetchall():

```

```

        store = {
            'store_id': row[0],
            'cycle_days': row[1],
            'avg_sales_multiplier':
float(row[2])
        }
        self.stores.append(store)

        # Initialize random start date
        for cycle staggering
            if row[0] not in
self.store_start_dates:
                offset_days =
random.randint(0, row[1] - 1)
                self.store_start_dates[row
[0]] = self.start_date +
timedelta(days=offset_days)

        logger.info(f"✓ Loaded
{len(self.stores):,} stores")

        def disable_indexes(self):
            """Drop indexes before bulk
insert"""
            logger.info("Dropping indexes for
faster insertion...")
            self.cursor.execute("DROP INDEX IF
EXISTS idx_orders_date")
            self.cursor.execute("DROP INDEX IF
EXISTS idx_orders_store")
            self.cursor.execute("DROP INDEX IF
EXISTS idx_orders_product")
            self.cursor.execute("DROP INDEX IF
EXISTS idx_orders_store_date")
            self.conn.commit()
            logger.info("✓ Indexes dropped")

        def enable_indexes(self):
            """Recreate indexes after bulk
insert"""
            logger.info("Recreating
indexes...")
            self.cursor.execute("CREATE INDEX
idx_orders_date ON orders(order_date)")
            self.cursor.execute("CREATE INDEX
idx_orders_store ON orders(store_id)")
            self.cursor.execute("CREATE INDEX
idx_orders_product ON orders(product_id)")
            self.cursor.execute("CREATE INDEX
idx_orders_store_date ON orders(store_id,
order_date)")
            self.conn.commit()
            logger.info("✓ Indexes
recreated")

```

```

        def generate_orders(self, resume: bool
= False):
            """Main order generation loop"""
            if resume:
                if self._load_checkpoint():
                    start_date =
self.last_processed_date +
timedelta(days=1)
                else:
                    logger.info("No checkpoint
found, starting from beginning")
                    start_date =
self.start_date
            else:
                start_date = self.start_date

            logger.info(f"Generating orders
from {start_date} to {self.end_date}")
            logger.info(f"Orders per cycle
event: {self.orders_per_cycle}")

            current_date = start_date
            batch_orders = []
            commit_interval = 1 # Commit
every 1 day for low disk space
            last_commit_date = current_date

            with tqdm(total=self.total_days,
initial=(current_date -
self.start_date).days) as pbar:
                while current_date <=
self.end_date:
                    date_orders = []

                    # Check each store for
cycle event
                    for store in self.stores:
                        store_id =
store['store_id']
                        cycle_days =
store['cycle_days']
                        multiplier =
store['avg_sales_multiplier']

                        store_start =
self.store_start_dates[store_id]
                        days_since_start =
(current_date - store_start).days

                        # Check if this is a
cycle event day
                        if days_since_start >=
0 and days_since_start % cycle_days == 0:

```

```

        # Generate orders
for this cycle event
        lambda_base = 8
        effective_lambda =
lambda_base * multiplier

        for _ in
range(self.orders_per_cycle):
            # Sample
product
            product_id =
random.choice(self.product_ids)
            unit_price =
float(self.product_prices[product_id])

            # Sample
quantity using Poisson distribution
            quantity =
max(1,
int(np.random.poisson(effective_lambda)))

            # Calculate
total value with price variance
            discount_multi
plier = random.uniform(0.85, 1.15)
            total_value =
round(quantity * unit_price *
discount_multiplier * multiplier, 2)

            date_orders.ap
pend((
                store_id,
                product_id
            ,
                current_da
te,
                quantity,
                total_valu
e
            ))

        # Bulk insert date orders
        if date_orders:
            insert_query = """
                INSERT INTO orders
            (store_id, product_id, order_date,
            quantity, total_value)
                VALUES %s
            """
            execute_values(self.cu
rsor, insert_query, date_orders,
            page_size=10000)
            self.total_orders_inse
rted += len(date_orders)

```

```

        # Commit periodically
        if (current_date -
last_commit_date).days >= commit_interval:
            self.conn.commit()
            self._save_checkpoint(
            )

            last_commit_date =
current_date

            self.last_processed_date =
current_date
            current_date +=
timedelta(days=1)
            pbar.update(1)
            pbar.set_postfix({
                'orders':
f'{self.total_orders_inserted:,}',
                'rate':
f'{self.total_orders_inserted /
((current_date - start_date).days +
1):.0f}/day'
            })

        # Final commit
        self.conn.commit()
        self._save_checkpoint()
        logger.info(f"✓ Order generation
complete: {self.total_orders_inserted:,}
orders inserted")

        def validate_orders(self):
            """Validate generated orders"""
            logger.info("Validating
orders...")

            # Total count
            self.cursor.execute("SELECT
COUNT(*) FROM orders")
            total = self.cursor.fetchone()[0]
            logger.info(f" Total orders:
{total:,}")

            # Date range
            self.cursor.execute("SELECT
MIN(order_date), MAX(order_date) FROM
orders")
            min_date, max_date =
self.cursor.fetchone()
            logger.info(f" Date range:
{min_date} to {max_date}")

            # Store type averages
            self.cursor.execute("""

```

```

        SELECT s.store_type,
        AVG(o.total_value) as avg_value
        FROM orders o
        JOIN stores s ON o.store_id =
        s.store_id
        GROUP BY s.store_type
        ORDER BY avg_value DESC
        """
        logger.info(" Average order value
        by store type:")
        for row in self.cursor.fetchall():
            logger.info(f" {row[0]}:
            ₹{row[1]:.2f}")

        logger.info("✓ Validation
        complete")

    def close(self):
        """Close database connection"""
        if self.cursor:
            self.cursor.close()
        if self.conn:
            self.conn.close()
        logger.info("Database connection
        closed")

def main():
    parser =
    argparse.ArgumentParser(description='Gener
    ate FMCG orders at billion scale')
    parser.add_argument('--db-config',
    default='db_config.json', help='Database
    config')
    parser.add_argument('--orders-per-
    cycle', type=int, default=200,
                        help='Orders per
    cycle event (default: 200 for ~1B
    orders)')
    parser.add_argument('--disable-
    indexes', action='store_true',
                        help='Drop indexes
    before insertion')
    parser.add_argument('--enable-
    indexes', action='store_true',
                        help='Recreate
    indexes after insertion')
    parser.add_argument('--resume',
    action='store_true',
                        help='Resume from
    checkpoint')
    parser.add_argument('--validate',
    action='store_true',

```

```

                        help='Validate
    orders after generation')

    args = parser.parse_args()

    # Load database config
    with open(args.db_config, 'r') as f:
        db_config = json.load(f)

    generator = OrderGenerator(db_config,
    args.orders_per_cycle)

    try:
        generator.connect_database()
        generator.load_products()
        generator.load_stores()

        if args.disable_indexes:
            generator.disable_indexes()

        generator.generate_orders(resume=a
    rgs.resume)

        if args.enable_indexes:
            generator.enable_indexes()

        if args.validate:
            generator.validate_orders()

    finally:
        generator.close()

    logger.info("✓ All operations
    complete!")

if __name__ == '__main__':
    main()

```

Transaction.py

```

"""
Transaction Generator - Creates 1:1
correspondence with orders table
"""

import json
import argparse
import logging
from typing import Dict

import psycopg2
from psycopg2.extras import execute_values

```

```

from tqdm import tqdm

logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s -
%(levelname)s - %(message)s',
                    handlers=[logging.FileH
andler('transaction_generator.log'),
logging.Stream
Handler())])
logger = logging.getLogger(__name__)

class TransactionGenerator:
    """Generate transactions with 1:1
order correspondence"""

    def __init__(self, db_config: Dict,
batch_size: int = 50000):
        self.db_config = db_config
        self.batch_size = batch_size
        self.conn = None
        self.cursor = None

    def connect_database(self):
        """Connect to PostgreSQL"""
        self.conn =
psycopg2.connect(**self.db_config)
        self.cursor = self.conn.cursor()
        logger.info("✓ Database
connected")

    def generate_transactions(self):
        """Generate transactions from
orders table"""
        logger.info("Generating
transactions...")

        # Get total order count
        self.cursor.execute("SELECT
COUNT(*) FROM orders")
        total_orders =
self.cursor.fetchone()[0]
        logger.info(f"Total orders to
process: {total_orders:,}")

        # Check existing transactions to
resume
        self.cursor.execute("SELECT
COUNT(*) FROM transactions")
        existing_count =
self.cursor.fetchone()[0]
        logger.info(f"Existing
transactions: {existing_count:,}")

```

```

        if existing_count >= total_orders:
            logger.info("All transactions
already generated!")
            return

        # Stream process orders in
batches, starting from where we left off
        offset = existing_count
        transaction_id = existing_count +
1

        logger.info(f"Resuming from
transaction {transaction_id:,}")

        with tqdm(total=total_orders) as
pbar:
            while offset < total_orders:
                # Fetch batch of orders
                self.cursor.execute("""
                SELECT order_id,
store_id, product_id, order_date,
quantity, total_value
                FROM orders
                ORDER BY order_id
                LIMIT %s OFFSET %s
                """, (self.batch_size,
offset))

                orders =
self.cursor.fetchall()
                if not orders:
                    break

                # Prepare transaction
records
                transactions = [
                    (transaction_id + i,
                     order[0], # order_id
                     order[1], # store_id
                     order[2], #
product_id
                     order[3], # date
                     order[4], # quantity
                     order[5], # value
                     round(float(order[5])
/ order[4], 2)) # unit_price
                    for i, order in
enumerate(orders)
                ]

                # Bulk insert
                insert_query = """
                INSERT INTO
transactions

```

```

            (transaction_id,
            order_id, store_id, product_id, date,
            quantity, value, unit_price)
            VALUES %s
        """
        execute_values(self.cursor
, insert_query, transactions)
        self.conn.commit()

        transaction_id +=
len(transactions)
        offset += self.batch_size
        pbar.update(len(orders))

        logger.info(f"✓ Generated
{transaction_id - 1:,} transactions")

        def validate_transactions(self):
            """Validate transaction
generation"""
            logger.info("Validating
transactions...")

            # Count match
            self.cursor.execute("SELECT
COUNT(*) FROM transactions")
            trans_count =
self.cursor.fetchone()[0]
            self.cursor.execute("SELECT
COUNT(*) FROM orders")
            order_count =
self.cursor.fetchone()[0]

            logger.info(f" Transactions:
{trans_count:,}")
            logger.info(f" Orders:
{order_count:,}")

            if trans_count == order_count:
                logger.info(" ✓ Count match
confirmed")
            else:
                logger.error(f" ✗ Count
mismatch: {abs(trans_count -
order_count):,} difference")

            # Check for orphaned transactions
            self.cursor.execute("""
                SELECT COUNT(*)
                FROM transactions t
                LEFT JOIN orders o ON
t.order_id = o.order_id
                WHERE o.order_id IS NULL
            """)

```

```

        orphaned =
self.cursor.fetchone()[0]

        if orphaned == 0:
            logger.info(" ✓ No orphaned
transactions")
        else:
            logger.error(f" ✗ Found
{orphaned:,} orphaned transactions")

        logger.info("✓ Validation
complete")

        def close(self):
            """Close connection"""
            if self.cursor:
                self.cursor.close()
            if self.conn:
                self.conn.close()

        def main():
            parser =
argparse.ArgumentParser(description='Gener
ate FMCG transactions')
            parser.add_argument('--db-config',
default='db_config.json')
            parser.add_argument('--batch-size',
type=int, default=50000)
            parser.add_argument('--validate',
action='store_true')

            args = parser.parse_args()

            with open(args.db_config, 'r') as f:
                db_config = json.load(f)

            generator =
TransactionGenerator(db_config,
args.batch_size)

            try:
                generator.connect_database()
                generator.generate_transactions()

                if args.validate:
                    generator.validate_transaction
s()
            finally:
                generator.close()

            logger.info("✓ Transaction generation
complete!")

```

```
if __name__ == '__main__':
    main()
```

Validate.py

```
#!/usr/bin/env python3
"""
FMCG Data Validation Script
Comprehensive data quality checks for
billion-scale dataset
"""

import json
import argparse
import logging
from typing import Dict
from datetime import date

import psycopg2

logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

class DataValidator:
    """Comprehensive validation for FMCG
    synthetic dataset"""

    def __init__(self, db_config: Dict,
                 brand_mapping_file: str):
        self.db_config = db_config
        with open(brand_mapping_file, 'r')
as f:
            data = json.load(f)
            self.brand_mappings =
data['brand_category_mappings']

        self.conn = None
        self.cursor = None
        self.errors = []

    def connect(self):
        """Connect to database"""
        self.conn =
psycopg2.connect(**self.db_config)
        self.cursor = self.conn.cursor()
        logger.info("✓ Database
connected")

    def validate_row_counts(self):
        """Validate table row counts"""
```

```
        logger.info("\n=== Row Count
Validation ===")

        tables = ['products', 'stores',
'orders', 'transactions']
        for table in tables:
            self.cursor.execute(f"SELECT
COUNT(*) FROM {table}")
            count =
self.cursor.fetchone()[0]
            logger.info(f" {table}:
{count:,} rows")

            if table == 'products' and
count < 100000:
                self.errors.append(f"Produ
cts count {count:,} below expected 100K+")
            elif table == 'orders' and
count < 900000000:
                self.errors.append(f"Order
s count {count:,} below target ~1
billion")

        def
validate_referential_integrity(self):
            """Check foreign key
relationships"""
            logger.info("\n=== Referential
Integrity Validation ===")

            # Orders -> Stores
            self.cursor.execute("""
                SELECT COUNT(*) FROM orders o
                LEFT JOIN stores s ON
o.store_id = s.store_id
                WHERE s.store_id IS NULL
            """)
            orphaned =
self.cursor.fetchone()[0]
            if orphaned > 0:
                self.errors.append(f"Found
{orphaned:,} orders with invalid
store_id")
            else:
                logger.info(" ✓ All orders
have valid store_id")

            # Transactions -> Orders
            self.cursor.execute("""
                SELECT COUNT(*) FROM
transactions t
                LEFT JOIN orders o ON
t.order_id = o.order_id
                WHERE o.order_id IS NULL
```

```

        """
        orphaned =
self.cursor.fetchone()[0]
        if orphaned > 0:
            self.errors.append(f"Found
{orphaned:,} transactions with invalid
order_id")
        else:
            logger.info(" ✓ All
transactions have valid order_id")

    def validate_date_range(self):
        """Validate date coverage"""
        logger.info("\n=== Date Range
Validation ===")

        self.cursor.execute("SELECT
MIN(order_date), MAX(order_date) FROM
orders")
        min_date, max_date =
self.cursor.fetchone()

        expected_start = date(2023, 6, 1)
        expected_end = date(2025, 12, 31)

        logger.info(f" Orders date range:
{min_date} to {max_date}")

        if min_date != expected_start:
            self.errors.append(f"Min date
{min_date} != expected {expected_start}")
        if max_date != expected_end:
            self.errors.append(f"Max date
{max_date} != expected {expected_end}")

        if not self.errors:
            logger.info(" ✓ Date range
correct")

    def
validate_brand_category_mappings(self):
        """Validate brand-category
ownership"""
        logger.info("\n=== Brand-Category
Mapping Validation ===")

        violations = []
        for brand, allowed_categories in
self.brand_mappings.items():
            self.cursor.execute("""
                SELECT DISTINCT category
                FROM products
                WHERE brand = %s
            """, (brand,))

```

```

        actual_categories = [row[0]
for row in self.cursor.fetchall()]

        for cat in actual_categories:
            if cat not in
allowed_categories:
                violations.append(f"Br
and '{brand}' in unauthorized category
'{cat}'")

        if violations:
            self.errors.extend(violations[
:10])
            logger.error(f" X Found
{len(violations)} violations")
        else:
            logger.info(" ✓ All brand-
category mappings valid")

    def
validate_mobile_vs_supermarket(self):
        """Critical validation: Mobile
stores should have lower avg order
value"""
        logger.info("\n=== Mobile vs
Supermarket Validation ===")

        self.cursor.execute("""
            SELECT s.store_type,
            AVG(o.total_value) as avg_value
            FROM orders o
            JOIN stores s ON o.store_id =
s.store_id
            WHERE s.store_type IN
('Mobile', 'Supermarket')
            GROUP BY s.store_type
        """)

        results = {row[0]: row[1] for row
in self.cursor.fetchall()}

        if 'Mobile' in results and
'Supermarket' in results:
            mobile_avg = results['Mobile']
            supermarket_avg =
results['Supermarket']
            ratio = mobile_avg /
supermarket_avg

            logger.info(f" Mobile avg:
₹{mobile_avg:.2f}")
            logger.info(f" Supermarket
avg: ₹{supermarket_avg:.2f}")

```

```

        logger.info(f" Ratio:
{ratio:.2f}")

        if ratio > 0.50:
            self.errors.append(
                f"Mobile/Supermarket
ratio {ratio:.2f} exceeds 0.50 threshold"
            )
        else:
            logger.info(" ✓ Mobile
stores have appropriately lower order
values")

    def validate_psku_hierarchy(self):
        """Validate PSKU-SKU structure"""
        logger.info("\n=== PSKU-SKU
Hierarchy Validation ===")

        # Count PSKUs
        self.cursor.execute("SELECT
COUNT(DISTINCT psku_code) FROM products")
        psku_count =
self.cursor.fetchone()[0]
        logger.info(f" Total PSKUs:
{psku_count:,}")

        if psku_count < 5000:
            self.errors.append(f"PSKU
count {psku_count:,} below target 5,000")

        # Check SKUs per PSKU
        self.cursor.execute("""
SELECT psku_code, COUNT(*) as
sku_count

FROM products
GROUP BY psku_code
HAVING COUNT(*) < 2
""")

        invalid_pskus =
self.cursor.fetchall()
        if invalid_pskus:
            self.errors.append(f"Found
{len(invalid_pskus)} PSKUs with < 2 SKUs")
        else:
            logger.info(" ✓ All PSKUs
have minimum 2 SKUs")

        # Average SKUs per PSKU
        self.cursor.execute("""
SELECT AVG(sku_count)
FROM (
    SELECT COUNT(*) as
sku_count

```

```

FROM products
GROUP BY psku_code
) sub
""")
        avg_skus =
self.cursor.fetchone()[0]
        logger.info(f" Avg SKUs per PSKU:
{avg_skus:.1f}")

    def validate_data_quality(self):
        """Check for NULL values and
invalid data"""
        logger.info("\n=== Data Quality
Validation ===")

        # Check for NULLs in products
        self.cursor.execute("""
SELECT COUNT(*) FROM products
WHERE product_id IS NULL OR
category IS NULL
OR brand IS NULL OR psku IS
NULL OR sku IS NULL
OR unit_price IS NULL
""")
        null_count =
self.cursor.fetchone()[0]
        if null_count > 0:
            self.errors.append(f"Found
{null_count:,} products with NULL required
fields")

        # Check for invalid quantities
        self.cursor.execute("SELECT
COUNT(*) FROM orders WHERE quantity < 1")
        invalid =
self.cursor.fetchone()[0]
        if invalid > 0:
            self.errors.append(f"Found
{invalid:,} orders with quantity < 1")

        # Check for invalid prices
        self.cursor.execute("SELECT
COUNT(*) FROM orders WHERE total_value <=
0")
        invalid =
self.cursor.fetchone()[0]
        if invalid > 0:
            self.errors.append(f"Found
{invalid:,} orders with total_value <= 0")

        if not self.errors:
            logger.info(" ✓ No data
quality issues found")

```

```

def generate_summary_report(self):
    """Generate validation summary"""
    logger.info("\n=== Validation
Summary ===")

    if not self.errors:
        logger.info("✓✓✓ ALL
VALIDATIONS PASSED ✓✓✓")
        return True
    else:
        logger.error(f"XXX FOUND
{len(self.errors)} ERRORS XXX")
        for i, error in
enumerate(self.errors, 1):
            logger.error(f" {i}.
{error}")
        return False

def run_all_validations(self):
    """Execute all validation
checks"""
    logger.info("Starting
comprehensive validation...")

    self.validate_row_counts()
    self.validate_referential_integrit
y()

    self.validate_date_range()
    self.validate_brand_category_mappi
ngs()

    self.validate_mobile_vs_supermarke
t()

    self.validate_psku_hierarchy()
    self.validate_data_quality()

    return

self.generate_summary_report()

def close(self):
    """Close connection"""
    if self.cursor:
        self.cursor.close()
    if self.conn:
        self.conn.close()

def main():
    parser =
argparse.ArgumentParser(description='Valid
ate FMCG synthetic dataset')
    parser.add_argument('--db-config',
default='db_config.json')
    parser.add_argument('--brand-mapping',
default='brand_category_mapping.json')

```

```

args = parser.parse_args()

with open(args.db_config, 'r') as f:
    db_config = json.load(f)

    validator = DataValidator(db_config,
args.brand_mapping)

    try:
        validator.connect()
        success =
validator.run_all_validations()
        exit_code = 0 if success else 1
    finally:
        validator.close()

    exit(exit_code)

if __name__ == '__main__':
    main()

```

Setup_DB.py

```

#!/usr/bin/env python3
"""
Database Setup Script
Creates database, loads DDL schema, and
optionally loads data
"""

import json
import argparse
import logging
import subprocess
import sys

import psycopg2
from psycopg2 import sql

logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s -
%(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

def create_database(db_config: dict):
    """Create database if it doesn't
exist"""
    db_name = db_config['database']

    # Connect to default postgres database

```

```

conn_config = db_config.copy()
conn_config['database'] = 'postgres'

try:
    conn =
psycopg2.connect(**conn_config)
    conn.autocommit = True
    cursor = conn.cursor()

    # Check if database exists
    cursor.execute("SELECT 1 FROM
pg_database WHERE datname = %s",
(db_name,))
    exists = cursor.fetchone()

    if not exists:
        logger.info(f"Creating
database '{db_name}'...")
        cursor.execute(sql.SQL("CREATE
DATABASE {}").format(
            sql.Identifier(db_name)
        ))
        logger.info(f"✓ Database
'{db_name}' created")
    else:
        logger.info(f"Database
'{db_name}' already exists")

    cursor.close()
    conn.close()
    return True

except psycopg2.Error as e:
    logger.error(f"Error creating
database: {e}")
    return False

def load_ddl(db_config: dict, ddl_file:
str = 'sql.sql'):
    """Load DDL schema"""
    try:
        logger.info(f"Loading DDL from
{ddl_file}...")
        conn =
psycopg2.connect(**db_config)
        cursor = conn.cursor()

        with open(ddl_file, 'r') as f:
            ddl_sql = f.read()

        cursor.execute(ddl_sql)
        conn.commit()

```

```

        cursor.close()
        conn.close()
        logger.info("✓ DDL schema loaded
successfully")
        return True

    except Exception as e:
        logger.error(f"Error loading DDL:
{e}")
        return False

def load_csv_data(db_config: dict,
product_csv: str = None,
store_csv: str = None,
transaction_csv: str = None):
    """Load CSV data using COPY command"""
    try:
        conn =
psycopg2.connect(**db_config)
        cursor = conn.cursor()

        if product_csv:
            logger.info(f"Loading products
from {product_csv}...")
            with open(product_csv, 'r',
encoding='utf-8') as f:
                cursor.copy_expert(
                    """COPY products
(product_id, category, sub_category,
brand, sub_brand,
psku, psku_code,
sku, sku_code, weight_value, weight_unit,
count_value,
unit_price)
FROM STDIN WITH CSV
HEADER""",
                    f
                )
            logger.info("✓ Products
loaded")

        if store_csv:
            logger.info(f"Loading stores
from {store_csv}...")
            with open(store_csv, 'r',
encoding='utf-8') as f:
                cursor.copy_expert(
                    """COPY stores
(store_id, store_type, cycle_days,
avg_sales_multiplier,
city, region,
state, pin_code, area_type,
established_date)

```

```

                                FROM STDIN WITH CSV
HEADER""",
                                f
                                )
                                logger.info("✓ Stores
loaded")

                                if transaction_csv:
                                logger.info(f"Loading
transactions from {transaction_csv}...")
                                with open(transaction_csv,
'r', encoding='utf-8') as f:
                                cursor.copy_expert(
                                "COPY transactions
FROM STDIN WITH CSV HEADER",
                                f
                                )
                                logger.info("✓ Transactions
loaded")

                                conn.commit()
                                cursor.close()
                                conn.close()
                                return True

                                except Exception as e:
                                logger.error(f"Error loading CSV
data: {e}")
                                return False

def main():
    parser =
    argparse.ArgumentParser(description='Setup
FMCG database')
    parser.add_argument('--db-config',
default='db_config.json',
                                help='Database
configuration file')
    parser.add_argument('--create-db',
action='store_true',
                                help='Create
database')
    parser.add_argument('--load-ddl',
action='store_true',
                                help='Load DDL
schema')
    parser.add_argument('--ddl-file',
default='sql.sql',
                                help='DDL file to
load')
    parser.add_argument('--load-products',
help='Product CSV file to load')

```

```

    parser.add_argument('--load-stores',
help='Store CSV file to load')
    parser.add_argument('--load-
transactions', help='Transaction CSV file
to load')

    args = parser.parse_args()

    # Load database config
    try:
        with open(args.db_config, 'r') as
f:
            db_config = json.load(f)
            logger.info(f"Loaded database
config from {args.db_config}")
            except Exception as e:
            logger.error(f"Error loading
config: {e}")
            sys.exit(1)

    # Create database
    if args.create_db:
        if not create_database(db_config):
            sys.exit(1)

    # Load DDL
    if args.load_ddl:
        if not load_ddl(db_config,
args.ddl_file):
            sys.exit(1)

    # Load CSV data
    if args.load_products or
args.load_stores or
args.load_transactions:
        if not load_csv_data(db_config,
args.load_products,
                                args.load_store
s, args.load_transactions):
            sys.exit(1)

    logger.info("✓ Database setup
complete!")

if __name__ == '__main__':
    main()

```

Brand_Category_Mapping.json

```

{
    "brand_category_mappings": {
        "Nestlé": [

```

```

    "Food & Beverages"
  ],
  "Unilever": [
    "Food & Beverages",
    "Personal Care",
    "Home Care",
    "Snacks"
  ],
  "ITC": [
    "Food & Beverages",
    "Personal Care",
    "Home Care",
    "Snacks",
    "Baby & Childcare",
    "Health"
  ],
  "Procter & Gamble": [
    "Personal Care",
    "Home Care",
    "Baby & Childcare"
  ],
  "Johnson & Johnson": [
    "Baby & Childcare",
    "Health"
  ],
  "Britannia": [
    "Food & Beverages"
  ],
  "Godrej": [
    "Personal Care",
    "Home Care"
  ],
  "PepsiCo": [
    "Food & Beverages",
    "Snacks"
  ],
  "Parle": [
    "Food & Beverages",
    "Snacks"
  ],
  "Dabur": [
    "Health",
    "Personal Care"
  ]
],
"brand_details": {
  "Nestlé": {
    "sub_brands": [
      "Nescafe",
      "Maggi",
      "KitKat",
      "Munch",
      "Milkybar",
      "Bar One",

```

```

      "Polo",
      "Milo",
      "a+",
      "Everyday",
      "Milkmaid",
      "Cerelac",
      "Lactogen",
      "Koko Krunch"
    ],
    "focus": "Food and nutrition"
  },
  "Unilever": {
    "sub_brands": [
      "Dove",
      "Lux",
      "Lifebuoy",
      "Pears",
      "Hamam",
      "Pepsodent",
      "Closeup",
      "Pond's",
      "Vaseline",
      "Lakmé",
      "Surf Excel",
      "Rin",
      "Wheel",
      "Vim",
      "Domex",
      "Comfort",
      "Brooke Bond",
      "Taj Mahal",
      "Red Label",
      "Bru",
      "Kissan",
      "Knorr",
      "Horlicks",
      "Kwality Walls",
      "Cornetto"
    ],
    "focus": "Maximum diversification
across consumer goods, home care, foods,
frozen desserts"
  },
  "ITC": {
    "sub_brands": [
      "Aashirvaad",
      "Sunfeast",
      "Bingo",
      "YiPPee",
      "B Natural",
      "Candyman",
      "Mint-o",
      "Kitchens of India",
      "Fiama Di Wills",

```

```

        "Vivel",
        "Savlon",
        "Essenza Di Wills",
        "Engage",
        "Superia",
        "Classmate"
    ],
    "focus": "India's most diversified conglomerate"
  },
  "Procter & Gamble": {
    "sub_brands": [
      "Pantene",
      "Head & Shoulders",
      "Gillette",
      "Oral-B",
      "Old Spice",
      "Ariel",
      "Tide",
      "Downy",
      "Lenor",
      "Ambi Pur",
      "Pampers"
    ],
    "focus": "Beauty, grooming, fabric care, baby care"
  },
  "Johnson & Johnson": {
    "sub_brands": [
      "Johnson's Baby",
      "Band-Aid",
      "Listerine"
    ],
    "focus": "Baby care heritage and healthcare"
  },
  "Britannia": {
    "sub_brands": [
      "Good Day",
      "Marie Gold",
      "NutriChoice",
      "Tiger",
      "Bourbon",
      "Treat",
      "Little Hearts"
    ],
    "focus": "Bakery and dairy specialization"
  },
  "Godrej": {
    "sub_brands": [
      "Godrej No.1",
      "Cinthol",
      "Goodknight"

```

```

    ],
    "focus": "Personal wash and household insecticide"
  },
  "PepsiCo": {
    "sub_brands": [
      "Pepsi",
      "Mountain Dew",
      "Tropicana",
      "Quaker",
      "Lay's",
      "Kurkure",
      "Doritos"
    ],
    "focus": "Beverages and snacks"
  },
  "Parle": {
    "sub_brands": [
      "Parle-G",
      "Monaco",
      "Krackjack",
      "Hide & Seek"
    ],
    "focus": "Biscuits and confectionery"
  },
  "Dabur": {
    "sub_brands": [
      "Dabur Honey",
      "Dabur Chyawanprash",
      "Dabur Lal Tail",
      "Dabur Pudim Hara",
      "Dabur Amla",
      "Vatika",
      "Dabur Red",
      "Babool",
      "Meswak"
    ],
    "focus": "Ayurveda and herbal products"
  }
}

```

DB_Config.json

```

{
  "host": "localhost",
  "port": 5433,
  "database": "fmcg_db",
  "user": "postgres",
  "password": "Abzu#2005"
}

```